

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Revisão P1- SO

28.03.2026

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Apresentação da Disciplina e Ferramentas
[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Tópicos de Estudo: [Ref. Livro]

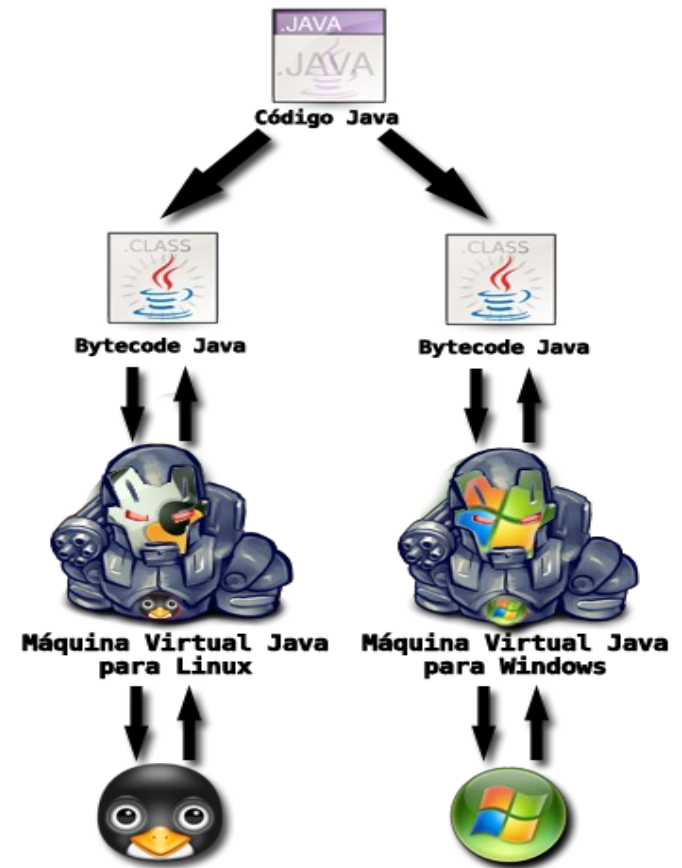
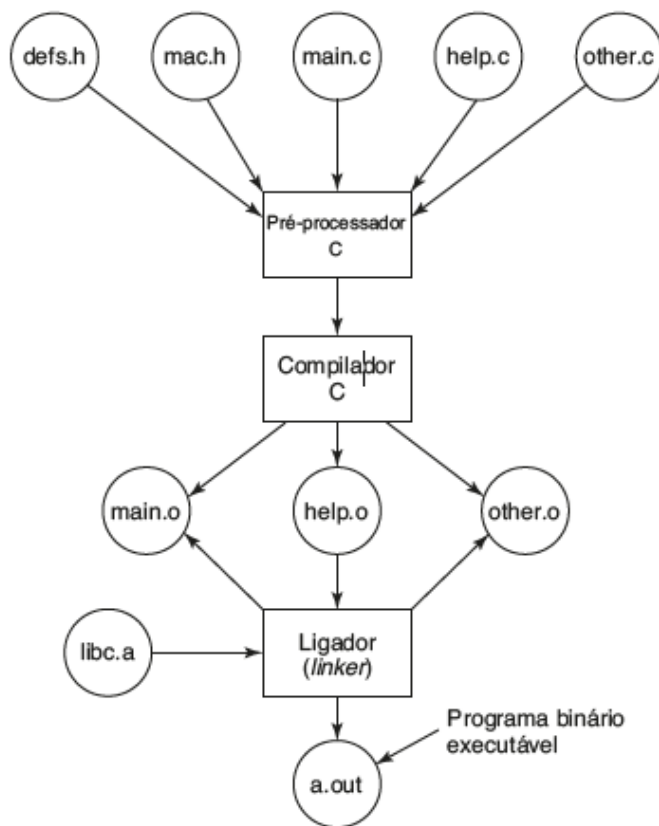
- 1) Apresentação da Disciplina
- 2) Introdução à Sistemas Operacionais [Capítulo 1]
- 3) Revisão sobre *hardware* de computadores [Capítulo 1]
- 4) Tipos e estruturas de SO [Capítulo 1]
- 5) Conceitos de Sistemas Operacionais [Capítulo 1]
- 6) Processos [Capítulo 2]
- 7) Escalonamento de processos [Capítulo 2]
- 8) *Threads* [Capítulo 2]
- 9) Comunicação entre processos [Capítulo 2]
- 10) Impasses (*Deadlocks*) [Capítulos 2 e 6]
- 11) Gerenciamento de Memória [Capítulo 3]
- 12) Memória Virtual [Capítulo 3]
- 13) Introdução ao Sistema de Arquivos [Capítulo 4]
- 14) Implementação dos Sistemas de Arquivos [Capítulo 4]
- 15) Entrada/Saída [Capítulo 5]

Por quê estudar SO?

Em ADS, SO é um conhecimento transversal!

→ **Compilado ou Interpretado, conceitos de SO se aplicam!** ←

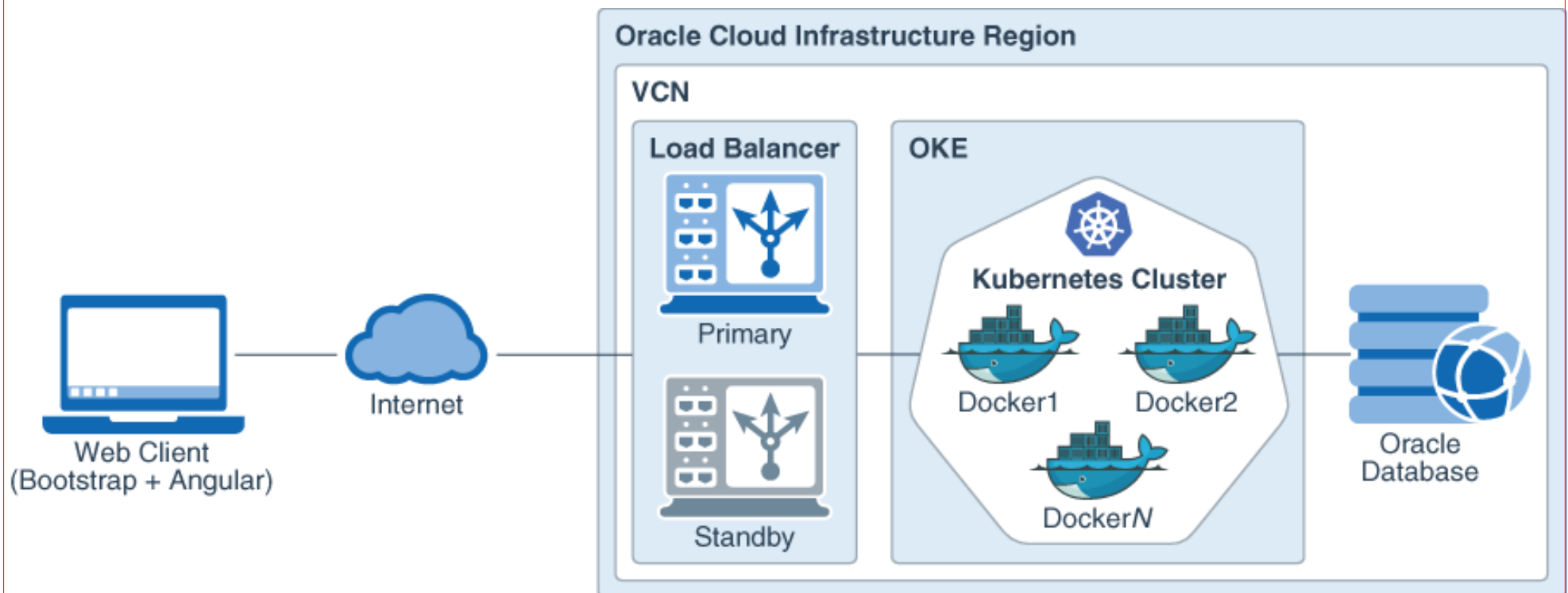
FIGURA 1.30 O processo de compilação de C e arquivos de cabeçalho para criar um arquivo executável.



Por quê estudar SO?

Em ADS, SO é um conhecimento transversal!

→ Fundamentos de SO também se aplicam à infraestrutura na nuvem ←



Por quê estudar SO?

Você, como programador, precisa ter uma fundação computacional sólida:

- **Aprenda bem os fundamentos da computação!**
- **Sistemas Operacionais, e, Arquitetura e Organização de Computadores, influenciam diretamente na programação.**

Relação com outras disciplinas

Arquitetura e Organização de Computadores:

Ex: Relação com memória

FIGURA 1.9 Uma hierarquia de memória típica. Os números são apenas aproximações.

Tempo típico de acesso		Capacidade típica
1 ns	Registradores	<1 KB
2 ns	Cache	4 MB
10 ns	Memória principal	1-8 GB
10 ms	Disco magnético	1-4 TB

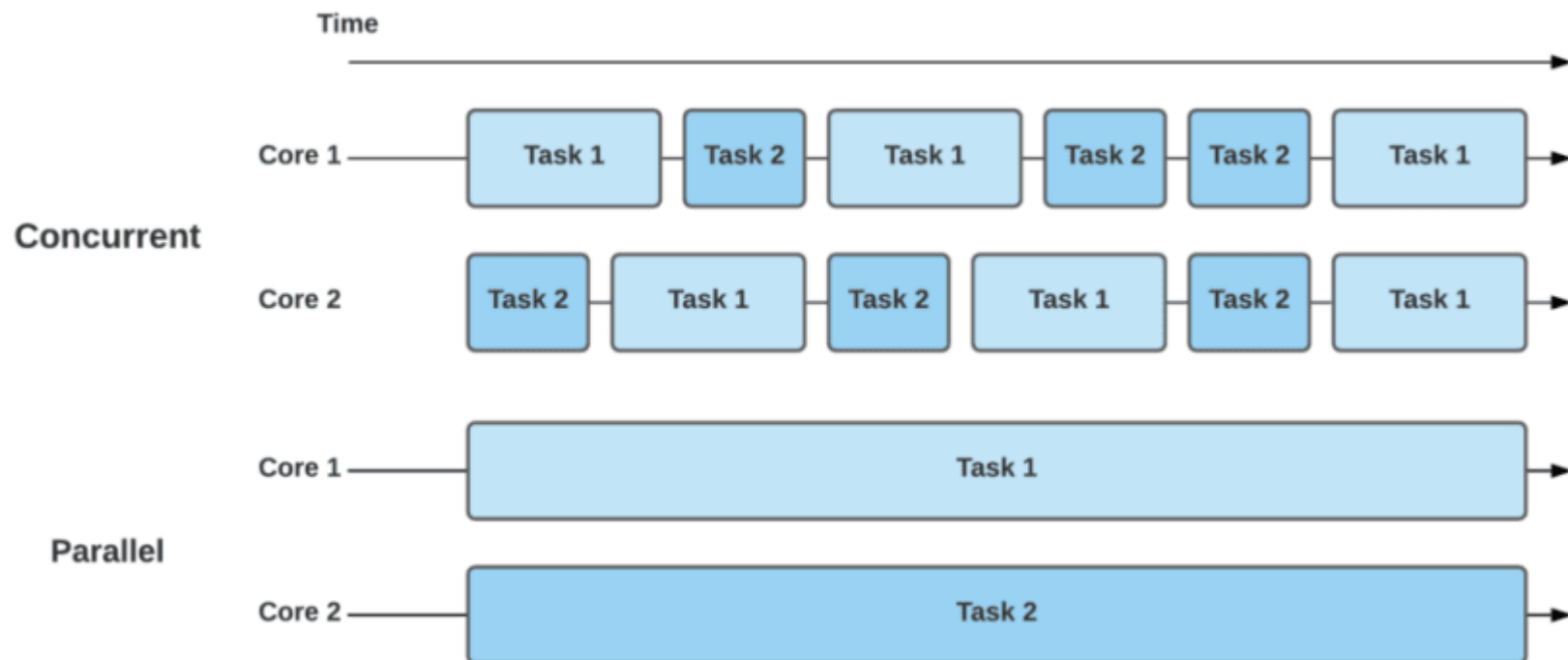
32 bits: Têm-se no máximo 4GB de memória RAM/Principal.

| $2^{10} \rightarrow$ kbytes | $2^{20} \rightarrow$ Mbytes | $2^{30} \rightarrow$ Gbytes |
31 (2^1) = 2 ... 32 (2^2) = 4

Relação com outras disciplinas

Algoritmos e Lógica de Programação:

Ex: Programação Concorrente e Paralela



Relação com outras disciplinas

Estruturas de Dados:

Conceitos aplicados nos SOs

- Pilha (*stack*)
- Fila (*queue*)
- Lista ligada/encadeada (*list*)
- Vetor (*array*)
- Matriz (*bitmap*)
- Escopo de função ou variável
- E outros...

Relação com outras disciplinas

Estruturas de Dados:

Linguagem C

“O mundo de acordo com a linguagem C”

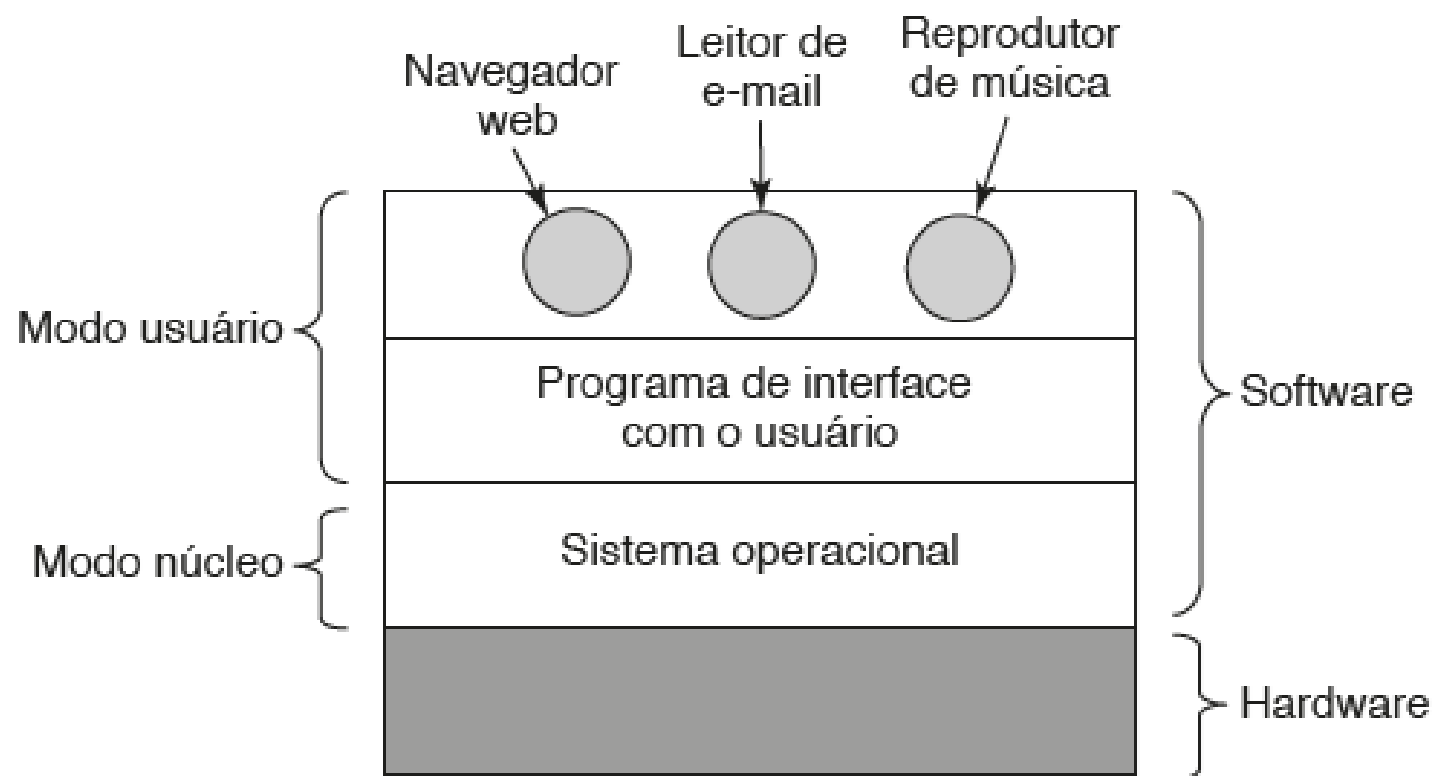
Sistemas operacionais normalmente são grandes programas C (ou C++) consistindo em muitas partes escritas por muitos programadores.

O ambiente usado para desenvolver os sistemas operacionais é muito diferente do que os indivíduos (tais como estudantes) estão acostumados quando estão escrevendo programas pequenos Java ou Python.

SO como Objeto de Estudo

O sistema operacional é a peça mais fundamental de software. **Opera diretamente sobre o hardware e proporciona a base para todos os outros softwares.** Seu posicionamento é mostrado na Figura 1.1.

FIGURA 1.1 Onde o sistema operacional se encaixa.



Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Introdução à Sistemas Operacionais
[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Sistema Computacional

Um computador moderno consiste em:

- um ou mais processadores
- memória principal
- discos (memória secundária)
- teclado, mouse, monitor, impressora, interface de rede e vários outros dispositivos de entrada e saída

Trata-se de um sistema complexo!

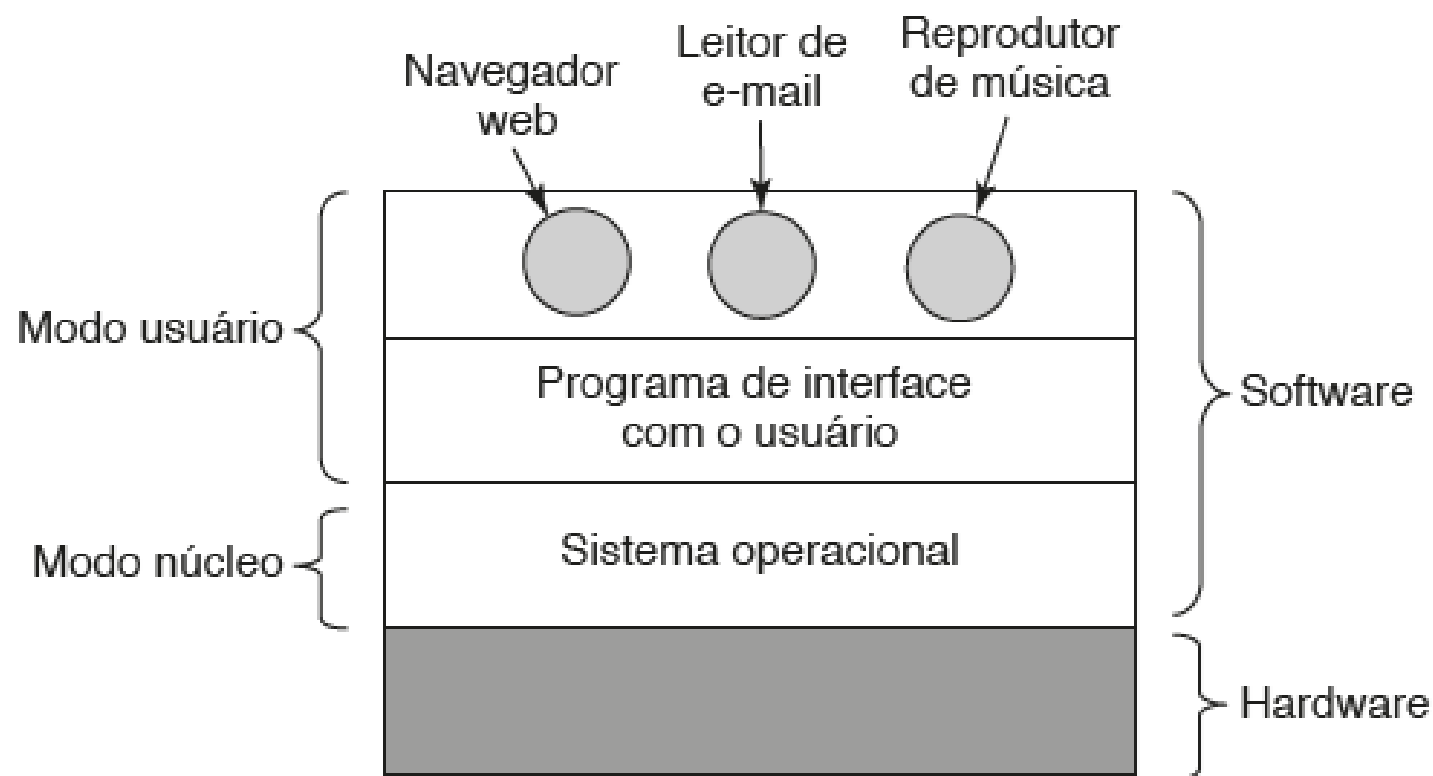
Gerenciar todos esses componentes e usá-los de maneira otimizada é um trabalho extremamente desafiador...

→ **Por essa razão, computadores são equipados com um dispositivo de software chamado de **Sistema Operacional**.**

Sistema Operacional

O sistema operacional é a peça mais fundamental de software. **Opera diretamente sobre o hardware e proporciona a base para todos os outros softwares.** Seu posicionamento é mostrado na Figura 1.1.

FIGURA 1.1 Onde o sistema operacional se encaixa.



Sistema Operacional

A maioria dos computadores tem dois modos de operação:

- modo núcleo
- e, modo usuário.

O **sistema operacional opera em modo núcleo** (também chamado modo supervisor).

Nesse modo ele tem acesso completo a todo o hardware e pode executar qualquer instrução que a máquina for capaz de executar!

O **resto do software opera em modo usuário**, no qual apenas um subconjunto das instruções da máquina está disponível.

→ **Instruções que afetam o controle da máquina ou realizam E/S (Entrada/Saída) são proibidas para programas de modo usuário!**

Temos uma definição objetiva de SO?

1.1 O que é um sistema operacional?

É difícil dizer com absoluta precisão o que é um sistema operacional, além de ele ser o software que opera em modo núcleo — e mesmo isso nem sempre é verdade. Parte do problema é que os sistemas operacionais realizam duas funções essencialmente não relacionadas: fornecer a programadores de aplicativos (e programas aplicativos, claro) um conjunto de recursos abstratos limpo em vez de recursos confusos de hardware, e gerenciar esses recursos de hardware. Dependendo de quem fala, você poderá ouvir mais a respeito de uma função do que de outra. Examinemos as duas então.

O que é um sistema operacional?

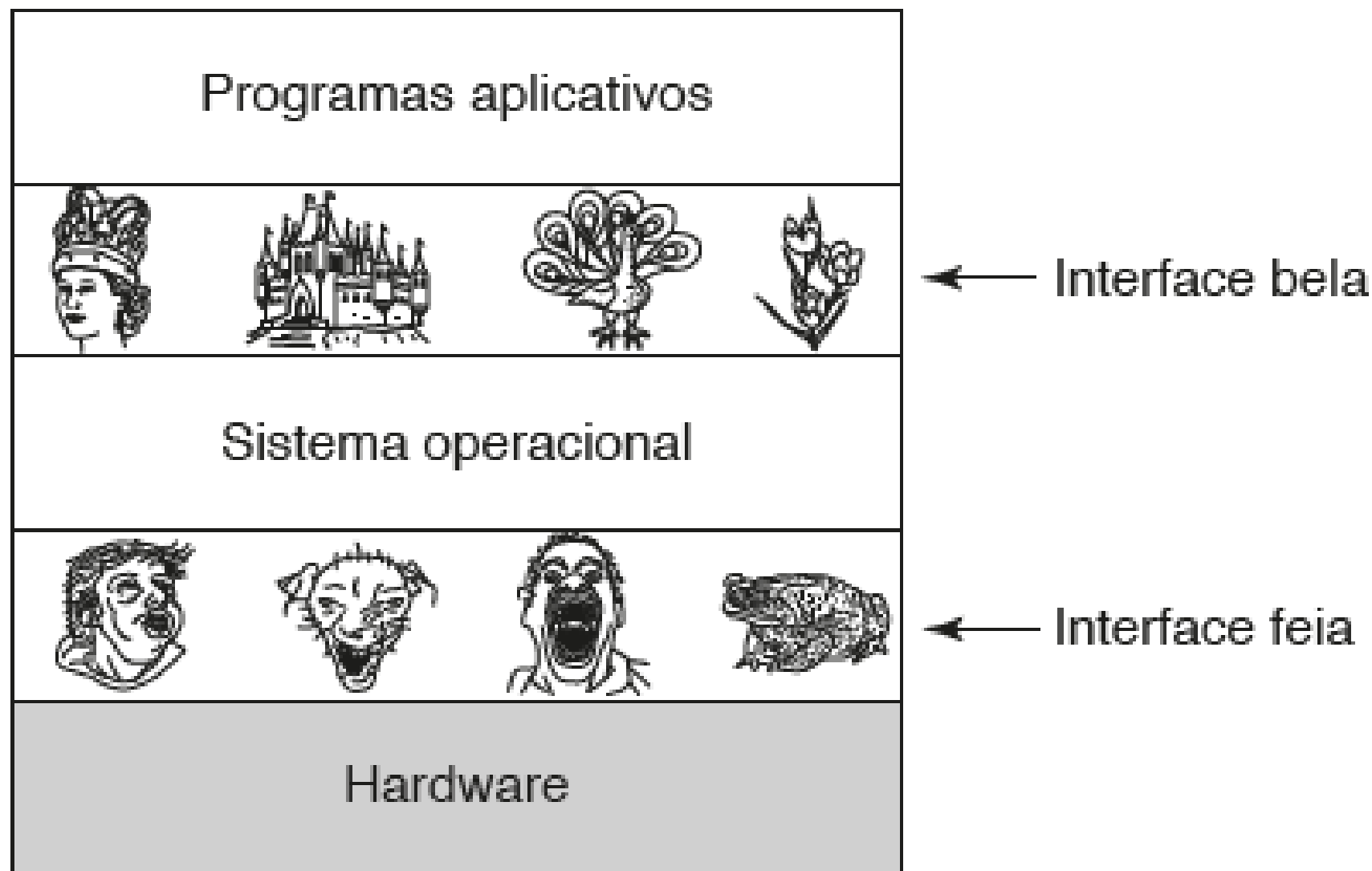
I) O sistema operacional como uma máquina estendida

→ Uma das principais tarefas dos sistemas operacionais é esconder o hardware e em vez disso apresentar abstrações de qualidade, limpas, elegantes, consistentes e fáceis de se trabalhar.

O conceito de um sistema operacional como fundamentalmente fornecendo abstrações para programas aplicativos é uma visão *top-down* (abstração de cima para baixo).

→ Sistemas operacionais transformam o feio em belo, como mostrado na Figura 1.2.

FIGURA 1.2 Sistemas operacionais transformam hardwares feios em belas abstrações.



O que é um sistema operacional?

II) O sistema operacional como um gerenciador de recursos

Na visão *bottom-up* (abstração de baixo para cima), a função do sistema operacional é fornecer uma alocação ordenada e controlada dos processadores, memórias e dispositivos de E/S entre os vários programas competindo por eles.

→ Esta visão sustenta que o sistema operacional está ali para gerenciar todas as partes do complexo sistema dos computadores modernos.

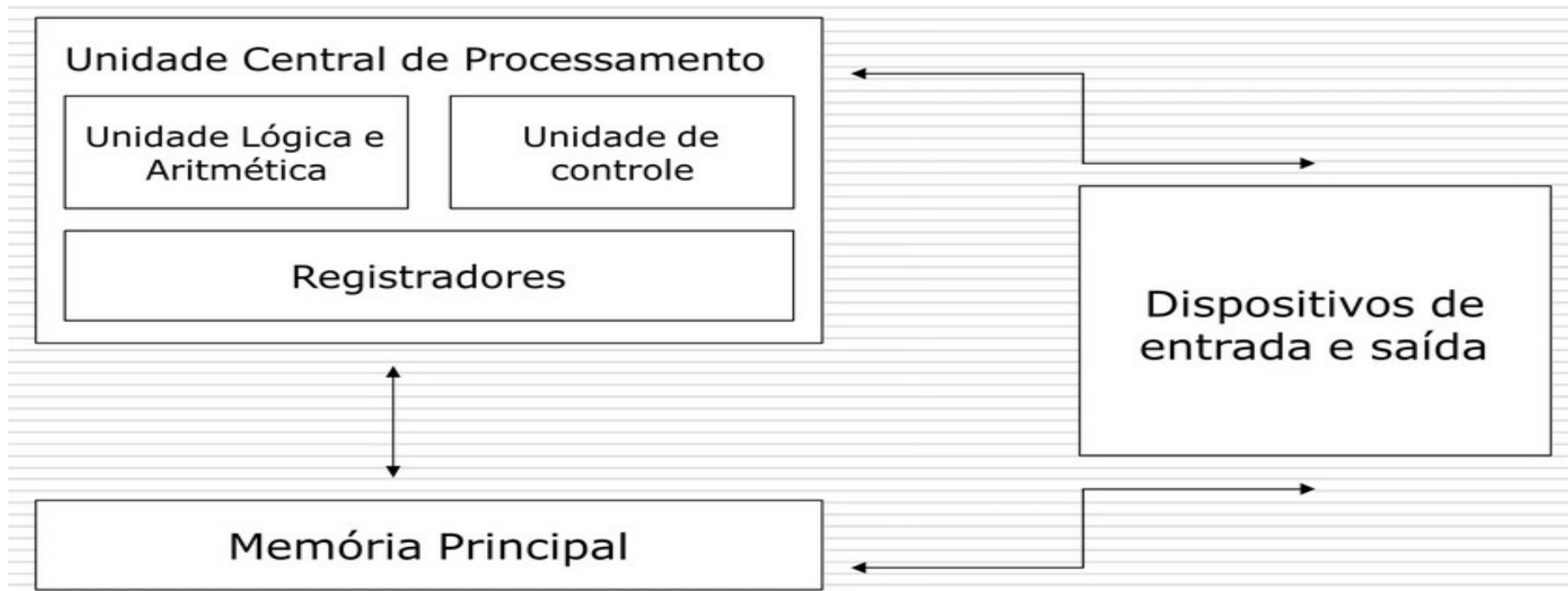
O gerenciamento de recursos inclui a **multiplexação** (compartilhamento) de recursos de duas maneiras diferentes: no tempo e no espaço.

→ **Tempo**: revezamento de diferentes programas ou usuários.

→ **Espaço**: cada cliente tem direito a uma parte do recurso.

Principais recursos gerenciados pelo SO:

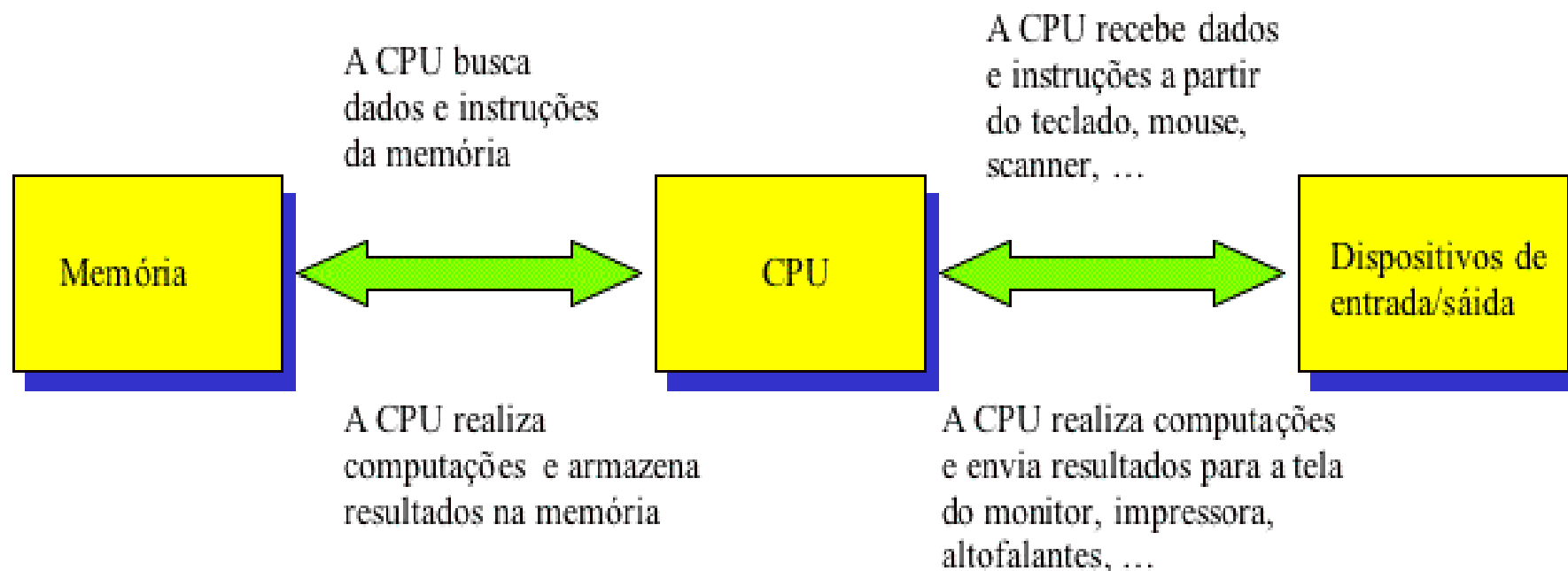
- Processador
- Memória
- Dispositivos de E/S



Arquitetura de von Neumann

Principais recursos gerenciados pelo SO:

- Processador
- Memória
- Dispositivos de E/S



Arquitetura de von Neumann

Interface

A interface é o principal programa de usuário.

Além de ser o principal programa de usuário, também é a parte mais percebida do SO. O ***shell*** (ou interpretador de comandos) é um programa de interface com o usuário baseado em texto. A ***GUI (Graphical User Interface)***, é um tipo de interface visual que utiliza ícones e menus gráficos.

A interface com o usuário é o nível mais inferior de software de modo usuário. Ela permite que ele inicie outros programas, como um navegador web, leitor de e-mail, ou reprodutor de música.

→ ***Shell*** e ***GUI*** não são parte do sistema operacional, mas usam-no ativamente para realizar o seu trabalho! Muitas vezes, usuários confundem a interface com o próprio sistema operacional. Ex: “tela preta” com Unix ou “*Windows Explorer*” com Windows.

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Revisão *Hardware* de Computadores
[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

CPU / Processador

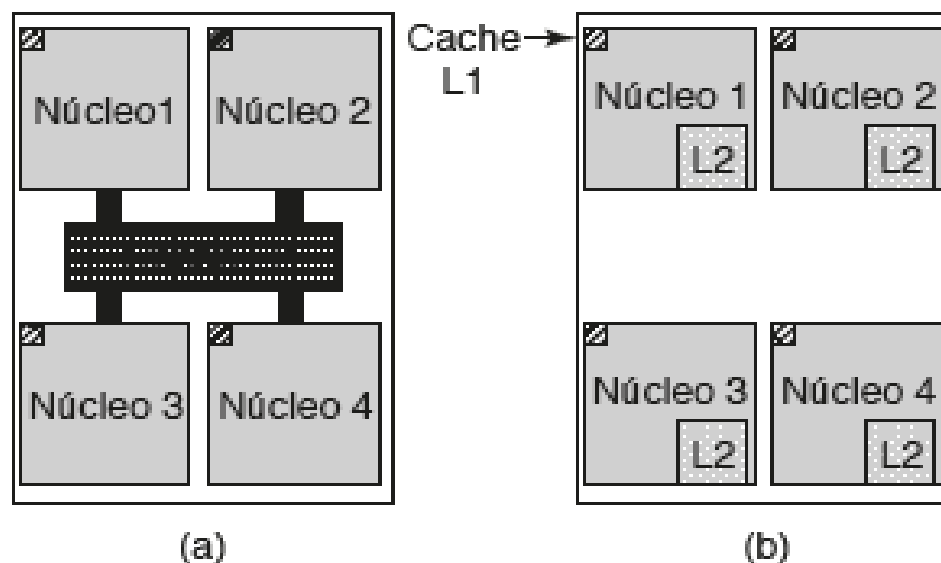
- **A CPU é o “cérebro” do computador é a CPU, responsável pelo ciclo básico de por buscar instruções na memória e executá-las.**
 - Este ciclo é repetido até o programa terminar.
 - É dessa maneira que os programas são executados.
- **Cada arquitetura/família de CPU tem um conjunto específico de instruções que ela consegue executar.**
 - Processador x86 não pode executar programas ARM.
 - Processador ARM não consegue executar programas x86.

CPU / Processador

- Chips Multithread e Multinúcleo

→ Os chips **Multinúcleo** tem vários processadores completos ou núcleos neles. Alguns processadores, como o Intel Xeon Phi apresentam mais de 60 núcleos em um único chip.

FIGURA 1.8 (a) Chip quad-core com uma cache L2 compartilhada.
(b) Um chip quad-core com caches L2 separadas.



→ **Multithreading** é um *fake* de multinúcleo, no qual o SO “enxerga” cada thread como uma CPU.

Memória

- **O segundo principal componente em qualquer computador.**
 - Idealmente, a memória perfeita, deve ser rápida, grande e muito barata.
 - Nenhuma tecnologia atual satisfaz esse ideal.
- **RAM (Random Access Memory) - memória de acesso aleatório.**
 - É chamada de memória principal e **possui comportamento volátil.**

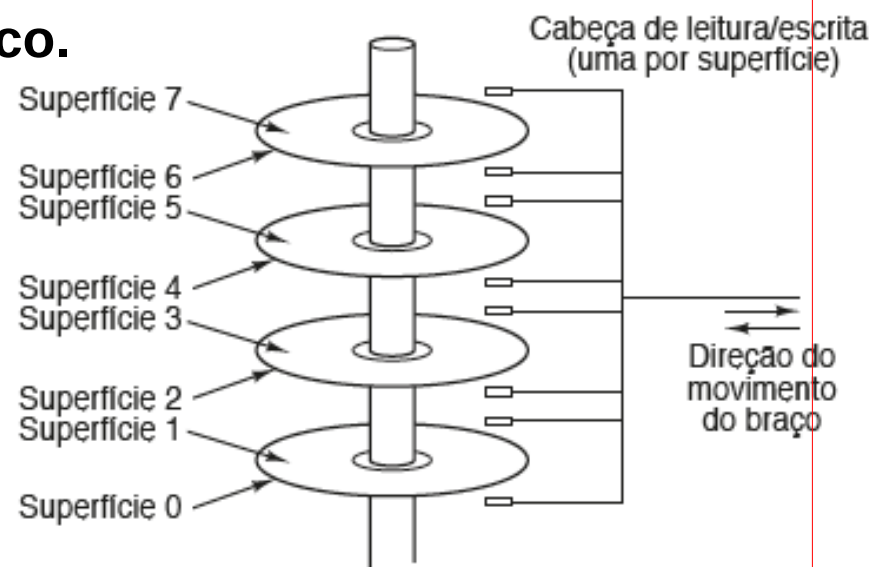
Discos

→ Chamados de memória secundária. **(comportamento não-volátil)**

- HDs (Hard Disk – disco rígido)

→ Dispositivo **mecânico** com disco magnético.

FIGURA 1.10 Estrutura de uma unidade de disco.

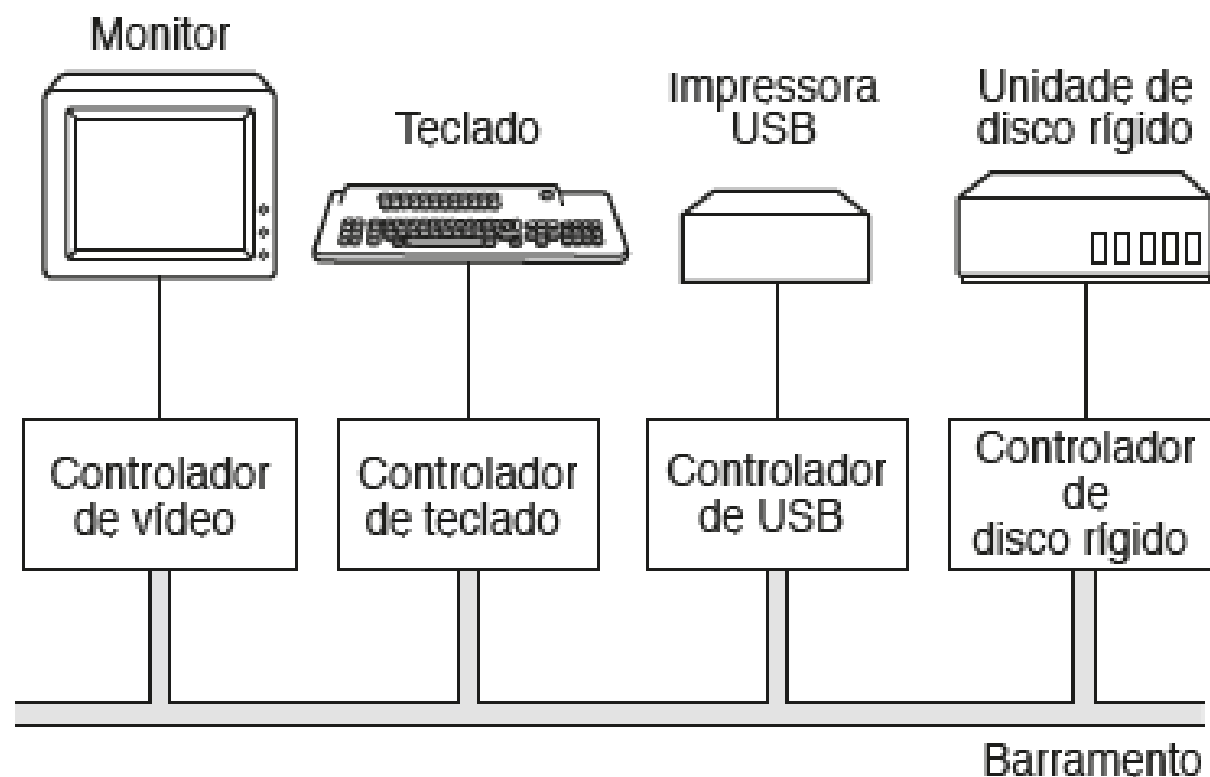


- SSDs (Solid State Disks – discos em estado sólido)

→ Conceitualmente, não são “discos”.

Dispositivos de Entrada e Saída

- Dispositivos de E/S também interagem intensamente com o SO.
- Geralmente consistem em duas partes.
 - Controlador
 - Dispositivo em si.



Inicialização do Computador

Resumidamente, o processo de inicialização funciona da seguinte maneira:

- Na *parentboard* (placa-pais) há um programa chamado de sistema **BIOS** (*Basic Input Output System* - sistema básico de entrada e saída).
 - O BIOS conta com rotinas de E/S de baixo nível, incluindo procedimentos para ler o teclado, escrever na tela e realizar a E/S no disco, entre outras coisas.
- Quando o computador é inicializado, o BIOS começa a executar...
 - Primeiro ele confere para ver quanta RAM está instalada e se o teclado e os outros dispositivos básicos estão instalados e respondendo corretamente.
 - Ele segue varrendo os barramentos para detectar todos os dispositivos ligados a ele.

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Tipos e Estruturas de SO

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Tipos de SO

Classificação pela finalidade de aplicação do SO

Zoológico dos Sistemas Operacionais

- Os sistemas operacionais existem há mais de meio século.
- Durante esse tempo, uma variedade bastante significativa deles foi desenvolvida.
 - Nem todos bastante conhecidos.

Abordaremos brevemente nove deles, considerando sua finalidade de aplicação.

A variedade de SOs que serão apresentados, estão organizados histórica e hierarquicamente:

- Dos mais antigos para os mais novos SOs;
- E, dos maiores para os menores sistemas computacionais.

Zoológico dos Sistemas Operacionais

1) Sistemas operacionais de computadores de grande porte (Mainframes)

- Um exemplo de sistema operacional de computadores de grande porte é o OS/390, um descendente do OS/360.
- No entanto, sistemas operacionais de computadores de grande porte estão pouco a pouco sendo substituídos por variantes UNIX.

Zoológico dos Sistemas Operacionais

2) Sistemas operacionais de servidores

- Eles são executados em servidores que são computadores pessoais muito grandes, em estações de trabalho ou mesmo computadores de grande porte.
- **Eles servem a múltiplos usuários ao mesmo tempo por meio de uma rede e permitem que os usuários compartilhem recursos de hardware e software.**
- Sistemas operacionais típicos de servidores são Solaris, FreeBSD, Linux e Windows Server.

Zoológico dos Sistemas Operacionais

3) Sistemas operacionais de multiprocessadores

- Com o advento recente de chips multinúcleo para computadores pessoais, mesmo sistemas operacionais de computadores convencionais estão começando a lidar com multiprocessadores em pequena escala.
- A parte difícil é fazer com que os **aplicativos de usuário** usem toda essa potência computacional.
- Muitos sistemas operacionais populares incluindo Windows e Linux, são executados em multiprocessadores.

Zoológico dos Sistemas Operacionais

4) Sistemas operacionais de computadores pessoais

- Todos os computadores modernos dão suporte à **multiprogramação**, muitas vezes com dezenas de programas iniciados no momento da inicialização do sistema.
 - Windows 11 tem cerca de 250 programas em execução quando inicia.
- O trabalho do SO de computadores pessoais é proporcionar um bom apoio para um único usuário.
 - Apoiando os programas de usuário, para o processamento de texto, planilhas e acesso à Internet.
- Exemplos comuns são o Linux, o Windows 7/8/10/11 e Mac OS da Apple.

Zoológico dos Sistemas Operacionais

5) Sistemas operacionais de computadores portáteis

- Seguindo com sistemas computacionais cada vez menores, chegamos aos **computadores portáteis** (*tablets, smartphones* e outros).
- Um computador portátil, é um computador pequeno que pode ser seguro na mão durante a operação.
- Os SOs dominantes neste mercado são o Android do Google e o iOS da Apple.

Zoológico dos Sistemas Operacionais

6) Sistemas operacionais embarcados

- Sistemas embarcados são executados em computadores que controlam dispositivos que não costumam ser vistos como computadores e que não aceitam softwares instalados pelo usuário.
- **A principal propriedade que distingue sistemas embarcados dos portáteis é a certeza de que nenhum software não confiável vá ser executado nele um dia.**
- Exemplos típicos são os fornos de micro-ondas, aparelhos multimídia em carros, sistemas de dispositivos como catracas ou relógios de ponto, sistemas de controle de componentes eletrônicos em motores de automóveis modernos.

Zoológico dos Sistemas Operacionais

7) Sistemas operacionais de nós sensores (*sensor-node*)

- Redes de nós sensores minúsculos estão sendo empregadas para uma série de finalidades.
- Esses nós são computadores minúsculos que se comunicam entre si e com uma estação-base usando comunicação sem fio.
- Os sensores são computadores pequenos movidos a bateria com rádios integrados. Eles têm energia limitada e precisam funcionar por longos períodos desacompanhados ao ar livre e frequentemente em condições severas.
- **Redes de sensores são usadas para** proteger os perímetros de prédios, pesquisas em biomas, medir a temperatura e a precipitação para a previsão de tempo, **colher informações do ambiente**, etc.

Zoológico dos Sistemas Operacionais

8) Sistemas operacionais de tempo real

- Esses sistemas são caracterizados por ter o tempo como um parâmetro-chave.
 - Por exemplo, em sistemas de controle de processo industrial, computadores em tempo real têm de coletar dados a respeito do processo de produção e usá-los para controlar máquinas na fábrica.
- **Se a ação tem de ocorrer absolutamente em um determinado momento, temos um sistema de tempo real crítico!**
- Em um sistema de tempo real **não-crítico**, embora não desejável, é **aceitável perder um prazo ocasional**, pois isso não causará danos permanentes. Exemplo: Sistemas de multimídia, *streaming*, e smartphones.

Zoológico dos Sistemas Operacionais

9) Sistemas operacionais de cartões inteligentes (*smartcard*)

- Uns dos menores sistemas operacionais são executados em cartões inteligentes, que são dispositivos do tamanho de cartões de crédito contendo um chip de CPU.
- **Possuem severas restrições de memória e processamento e de energia.**
 - Alguns obtêm energia por contatos no leitor ou por indução.
- Normalmente conseguem realizar somente uma função, como pagamentos eletrônicos, identificação de entidades relacionadas.
- Muitas vezes são sistemas proprietários.

Estruturas de SO

Classificação pela forma que o SO se organiza internamente.

Estrutura de Sistemas Operacionais

- Além da variedade de SOs apresentadas no Zoológico dos Sistemas Operacionais...
- Podemos **analisar os sistemas operacionais de acordo com sua estrutura.**
- Abordaremos brevemente seis arquiteturas, **considerando a forma que o SO se organiza internamente.**

IMPORTANTE:

→ As estruturas apresentadas **são de caráter exemplificativo**, com a finalidade de termos alguma ideia do espectro de possibilidades.

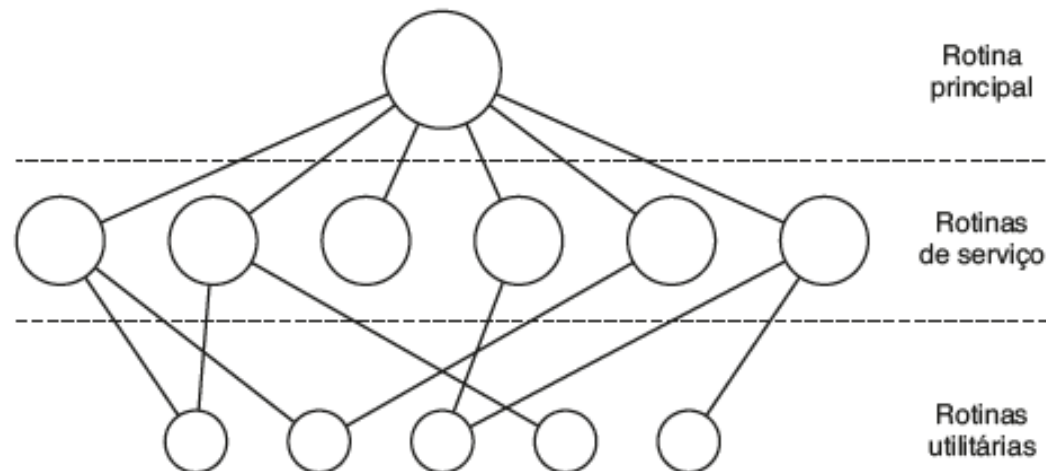
→ **Não** significa que estas **são as únicas** formas de organização possíveis.

Estrutura de Sistemas Operacionais

1) Sistemas monolíticos

- Todo o sistema operacional é executado como um único programa em modo núcleo, sendo todo o SO carregado quando o computador é inicializado. Ex: MS-DOS, Windows 95/98.
- Apesar de monolíticos, **podem conter certa estruturação, separando as tarefas da rotina principal, das tarefas de rotinas de serviço ou utilitárias.**

FIGURA 1.24 Um modelo de estruturação simples para um sistema monolítico.



Estrutura de Sistemas Operacionais

2) Sistemas de camadas

- Forma de organização na qual o sistema operacional pode ser visto como uma hierarquia de camadas, cada uma construída sobre a camada abaixo dela. O primeiro sistema construído dessa maneira foi o sistema THE, sistema holandês desenvolvido na Technische Hogeschool Eindhoven por E. W. Dijkstra et. al. (1968).

→ Organizado em seis camadas, o THE era um sistema em lote simples, desenvolvido para o computador Electrologica X8.

FIGURA 1.25 Estrutura do sistema operacional THE.

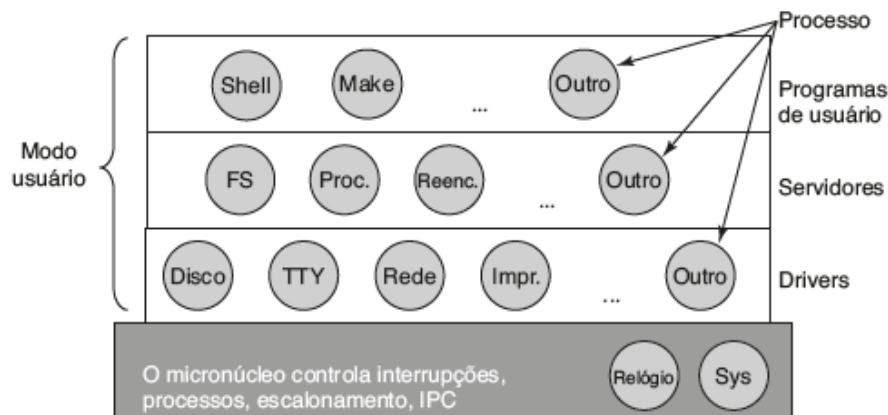
Camada	Função
5	O operador
4	Programas de usuário
3	Gerenciamento de entrada/saída
2	Comunicação operador-processo
1	Memória e gerenciamento de tambor
0	Alocação do processador e multiprogramação

Estrutura de Sistemas Operacionais

3) Micronúcleos

- A ideia básica por trás do projeto de micronúcleo é atingir uma alta confiabilidade através da **divisão do sistema operacional em módulos pequenos e bem definidos, dos quais apenas o micronúcleo é executado em modo núcleo** e o resto é executado como processos de usuário comuns relativamente sem poder.
- O objetivo é ter o mínimo possível sendo executado no modo núcleo, pois **erros no código do núcleo podem derrubar o sistema** instantaneamente.

FIGURA 1.26 Estrutura simplificada do sistema MINIX.

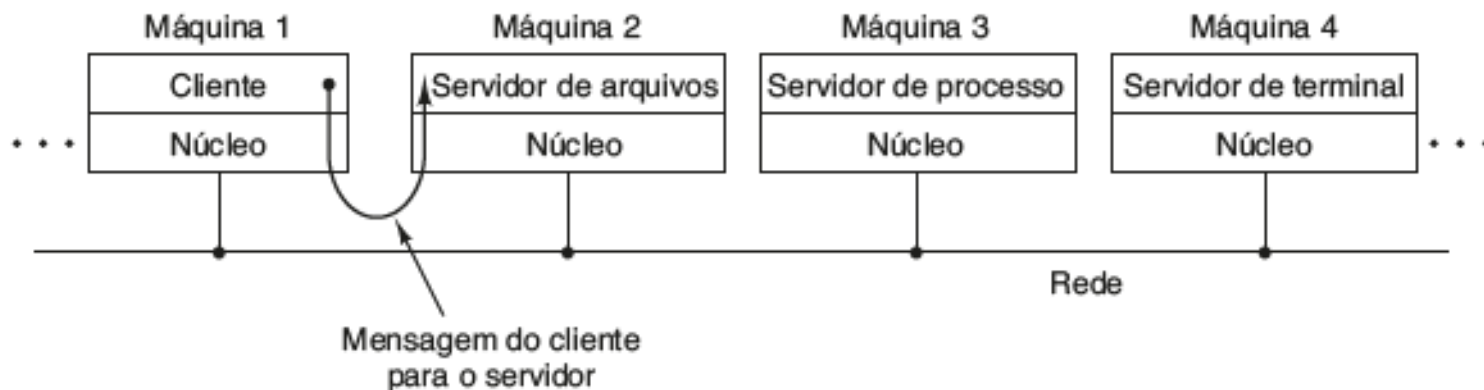


Estrutura de Sistemas Operacionais

4) O modelo cliente-servidor

- A essência do modelo encontra-se na presença de processos clientes e processos servidores. Os servidores, prestam algum serviço, e os clientes, que usam esses serviços.
- A comunicação entre clientes e servidores é realizada muitas vezes pela troca de mensagens. Os clientes não precisam saber se as mensagens são entregues localmente em suas próprias máquinas, ou se são enviadas através de uma rede para servidores em uma máquina remota.

FIGURA 1.27 O modelo cliente-servidor em uma rede.



Estrutura de Sistemas Operacionais

5) Máquinas virtuais

- A ideia da **virtualização** foi em grande parte ignorada no mundo dos PCs até há pouco tempo. Mas nos últimos anos, uma combinação de novas necessidades, novo software e novas tecnologias combinaram-se para torná-la um tópico de alto interesse.
- A execução de programas é outra área onde as máquinas virtuais são amplamente utilizadas.
 - Quando a Sun Microsystems inventou a linguagem de programação Java, ela também inventou uma máquina virtual (isto é, uma arquitetura de computadores) chamada de JVM (*Java Virtual Machine* — máquina virtual Java).
 - A Microsoft também criou uma máquina virtual para a plataforma .NET.

Estrutura de Sistemas Operacionais

6) Exonúcleos

- Uma **abordagem diferente para a virtualização**.
- Em vez de clonar a máquina real, como é feito com as máquinas virtuais, outra estratégia é dividi-la, oferecendo à cada usuário um subconjunto dos recursos.
 - Ex: Uma máquina virtual pode obter os blocos de disco de 0 a 1.023, a próxima pode ficar com os blocos 1.024 a 2.047 e assim por diante.
- **Cada máquina virtual no nível do usuário pode executar seu próprio sistema operacional.**

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Conceitos de Sistemas Operacionais

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Conceitos de sistemas operacionais

Antes, precisamos estabelecer dois tipos de sistemas operacionais, **considerando a classificação quanto ao compartilhamento de hardware:**

- **Sistema Operacional Monotarefa ou Monoprogramado**

→ Só executam um único programa por vez. O programa é carregado e fica em execução até seu encerramento, sem a possibilidade do usuário executar outro programa ao mesmo tempo. Ex: MS-DOS.

- **Sistema Operacional Multitarefa ou Multiprogramado**

→ Permite manter mais de um programa em execução, realizando o compartilhamento da CPU e demais recursos do computador. Ex: Windows 11.

→ **Os conceitos a seguir tem forte relação com SO Multiprogramado.**

Conceitos de sistemas operacionais

Determinados conceitos e abstrações básicas são fundamentais para compreender o funcionamento de sistemas operacionais. Tais como:

- Processos
- Espaços de endereços
- Arquivos
- Entrada/Saída
- Proteção
- O interpretador de comandos (*shell*)
- Interrupção
- Chamadas de sistema

Conceitos de sistemas operacionais

Processos

Um processo é basicamente um programa em execução. É um conceito fundamental em todos os sistemas operacionais.

Associado a cada processo está o espaço de endereçamento e um conjunto de recursos, em geral abrangendo registradores (incluindo o contador de programa e o ponteiro de pilha), uma lista de arquivos abertos, listas de processos relacionados e todas as demais informações necessárias para execução do programa.

→ **Um processo é na essência um contêiner que armazena todas as informações necessárias para executar um programa.** A maneira mais fácil de compreender intuitivamente um processo é pensar a respeito do sistema de multiprogramação.

Conceitos de sistemas operacionais

Espaços de endereçamento

Todo computador tem memória principal para armazenar programas em execução... **O espaço de endereçamento é uma lista de posições de memória que vai de 0 a algum máximo, onde o processo pode ler e escrever.**

→ **O espaço de endereçamento é desacoplado da memória física da máquina e pode ser maior ou menor do que a memória física.**

O espaço de endereçamento contém o programa executável, os dados do programa e sua pilha. **O gerenciamento de espaços de endereçamento e da memória física forma uma parte importante do que faz um sistema operacional.**

Conceitos de sistemas operacionais

Arquivos

Outro conceito fundamental que conta com o suporte de virtualmente todos os sistemas operacionais é o sistema de arquivos.

Arquivo é uma abstração que o SO utiliza para esconder as peculiaridades dos discos e outros dispositivos de E/S, apresentando um modelo agradável e claro de arquivos que sejam independentes dos dispositivos.

A maioria dos sistemas operacionais de PCs tem o **conceito de diretório como uma maneira de agrupar os arquivos.**

Conceitos de sistemas operacionais

Entrada/Saída

Todos os computadores têm **dispositivos físicos para obter entradas e produzir saídas**. Cabe ao sistema operacional gerenciá-los.

→ Existem muitos tipos de dispositivos de entrada e de saída, incluindo teclados, monitores, impressoras e assim por diante.

Em consequência, todo sistema operacional tem um subsistema de E/S para gerenciar os dispositivos de E/S.

Alguns softwares de E/S são independentes do dispositivo, isto é, aplicam-se igualmente bem a muitos ou a todos dispositivos de E/S. Outras partes dele, como *drivers* de dispositivo, são específicos a dispositivos de E/S particulares.

Conceitos de sistemas operacionais

Proteção

Computadores contêm grandes quantidades de **informações que os usuários muitas vezes querem proteger e manter confidenciais.**

Essas informações podem incluir e-mails, planos de negócios, declarações fiscais e muito mais. **Cabe ao sistema operacional gerenciar a segurança do sistema de maneira que os arquivos, por exemplo, sejam acessíveis somente por usuários autorizados.**

Além da proteção ao arquivo, há muitas outras questões de segurança.

→ Proteger o sistema de intrusos indesejados, humanos ou não (por exemplo, vírus) é uma delas.

Conceitos de sistemas operacionais

O interpretador de comandos (*shell*)

O interpretador de comandos é a principal interface entre um usuário e o sistema operacional, a não ser que o usuário esteja usando uma interface gráfica.

O *shell* é um programa de usuário que faz um uso intensivo de muitos aspectos do sistema operacional. Através do *shell* é possível invocar comandos que realizam chamadas de sistema, enquanto o sistema operacional é o código que executa as chamadas de sistema.

O *shell* tem o terminal como entrada-padrão e saída-padrão. Ele inicia emitindo um caractere de *prompt*, que diz ao usuário que o *shell* está esperando para aceitar um comando.

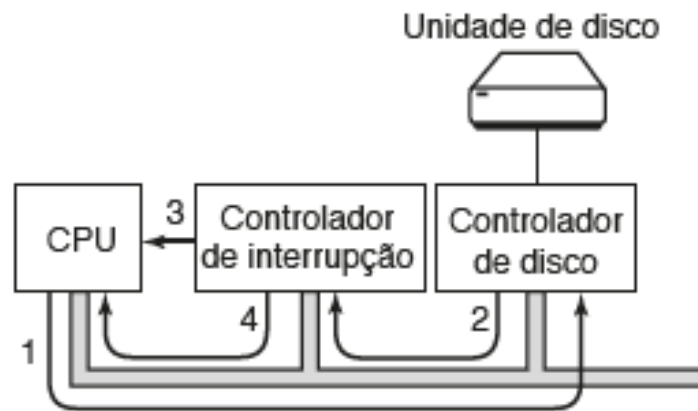
Interrupções

Interrupções são muito importantes em sistemas operacionais, **funcionam como alertas e indicativos para a ocorrência de eventos.**

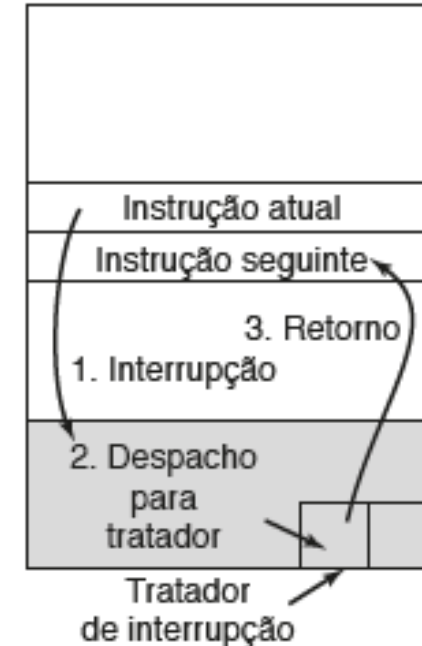


Interrupções

FIGURA 1.11 (a) Os passos para iniciar um dispositivo de E/S e obter uma interrupção. (b) O processamento de interrupção envolve obter a interrupção, executar o tratador de interrupção e retornar ao programa do usuário.



(a)



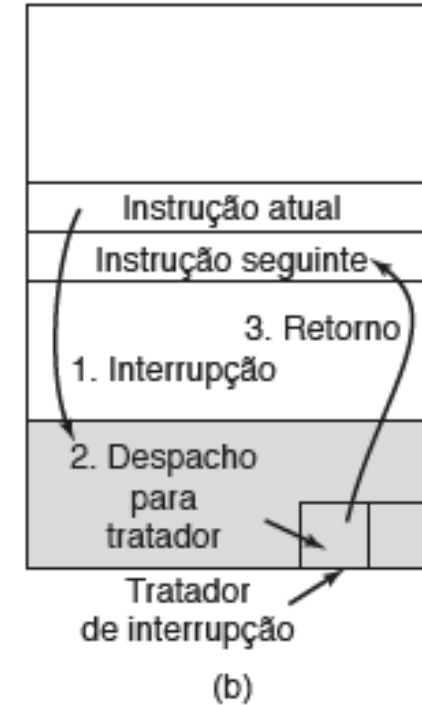
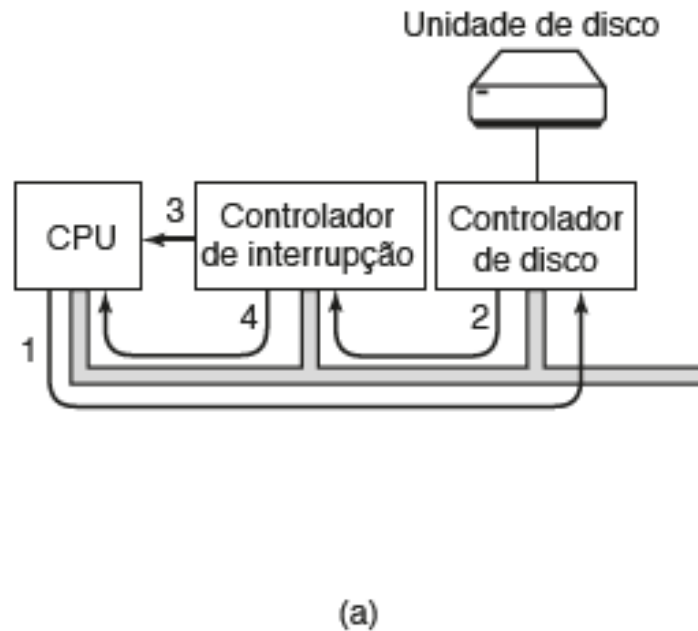
(b)

Um processo de quatro passos para a E/S:

No **passo 1**, o *driver* diz para o controlador o que fazer escrevendo nos seus registradores de dispositivo.

Interrupções

FIGURA 1.11 (a) Os passos para iniciar um dispositivo de E/S e obter uma interrupção. (b) O processamento de interrupção envolve obter a interrupção, executar o tratador de interrupção e retornar ao programa do usuário.

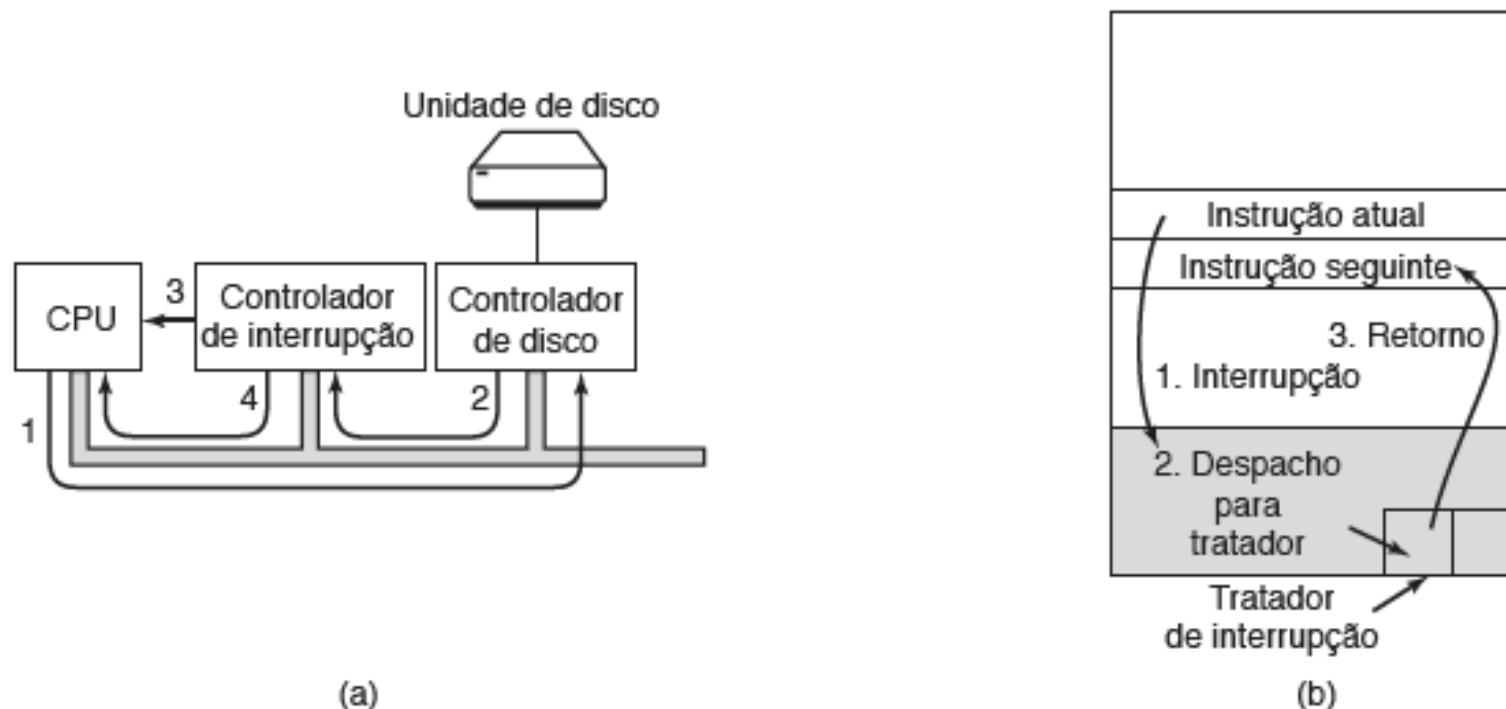


Um processo de quatro passos para a E/S:

Quando o controlador termina de ler ou escrever o número de bytes que lhe disseram para transferir, ele sinaliza o chip controlador de interrupção usando determinadas linhas de barramento no **passo 2**.

Interrupções

FIGURA 1.11 (a) Os passos para iniciar um dispositivo de E/S e obter uma interrupção. (b) O processamento de interrupção envolve obter a interrupção, executar o tratador de interrupção e retornar ao programa do usuário.

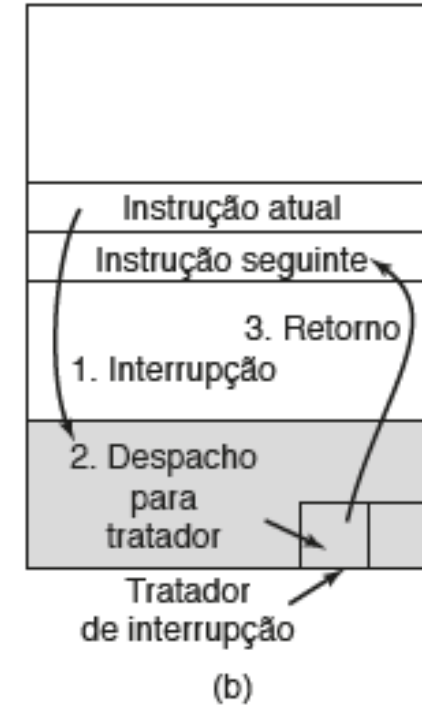
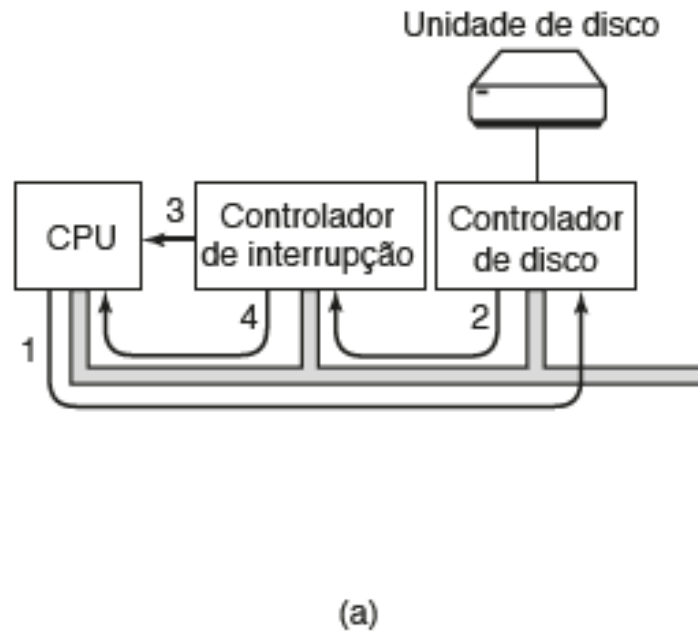


Um processo de quatro passos para a E/S:

Se o controlador de interrupção está pronto para aceitar a interrupção (o que ele talvez não esteja, se estiver ocupado lidando com uma interrupção de maior prioridade), ele sinaliza isso à CPU no **passo 3**.

Interrupções

FIGURA 1.11 (a) Os passos para iniciar um dispositivo de E/S e obter uma interrupção. (b) O processamento de interrupção envolve obter a interrupção, executar o tratador de interrupção e retornar ao programa do usuário.



Um processo de quatro passos para a E/S:

No **passo 4**, o controlador de interrupção insere o número do dispositivo no barramento de maneira que a CPU possa lê-lo e saber qual dispositivo acabou de terminar.

Chamadas de sistema

A interface entre os programas de usuários e o sistema operacional diz respeito fundamentalmente a abstrações. Para compreender de verdade o que os sistemas operacionais fazem, temos de examinar essa interface de perto.

As chamadas de sistemas são utilizadas para executar ações que requerem execução em modo núcleo do sistema operacional.

→ **Ações que afetam diretamente o hardware** (ex: alocação ou uso de recurso).

→ **Precisam de privilégios elevados!**

Apesar de ser um conceito comum e amplamente difundido, chamadas de sistema **variam de um sistema operacional para outro**, embora os conceitos aplicados tendam a ser similares.

Chamadas de sistema: Nos SOs.

FIGURA 1.18 Algumas das principais chamadas de sistema POSIX. O código de retorno *s* é -1 se um erro tiver ocorrido. Os códigos de retorno são os seguintes: *pid* é um processo id, *fd* é um descritor de arquivo, *n* é um contador de bytes, *position* é um deslocamento no interior do arquivo e *seconds* é o tempo decorrido. Os parâmetros são explicados no texto.

Gerenciamento de processos

Chamada	Descrição
<code>pid = fork()</code>	Cria um processo filho idêntico ao pai
<code>pid = waitpid(pid, &statloc, options)</code>	Espera que um processo filho seja concluído
<code>s = execve(name, argv, environp)</code>	Substitui a imagem do núcleo de um processo
<code>exit(status)</code>	Conclui a execução do processo e devolve status

Gerenciamento de arquivos

Chamada	Descrição
<code>fd = open(file, how, ...)</code>	Abre um arquivo para leitura, escrita ou ambos
<code>s = close(fd)</code>	Fecha um arquivo aberto
<code>n = read(fd, buffer, nbytes)</code>	Lê dados a partir de um arquivo em um buffer
<code>n = write(fd, buffer, nbytes)</code>	Escreve dados a partir de um buffer em um arquivo
<code>position = lseek(fd, offset, whence)</code>	Move o ponteiro do arquivo
<code>s = stat(name, &buf)</code>	Obtém informações sobre um arquivo

Gerenciamento do sistema de diretório e arquivo

Chamada	Descrição
<code>s = mkdir(name, mode)</code>	Cria um novo diretório
<code>s = rmdir(name)</code>	Remove um diretório vazio
<code>s = link(name1, name2)</code>	Cria uma nova entrada, name2, apontando para name1

Chamadas de sistema: Nos SOs.

FIGURA 1.23 As chamadas da API Win32 que correspondem aproximadamente às chamadas UNIX da Figura 1.18. Vale a pena enfatizar que o Windows tem um número muito grande de outras chamadas de sistema, a maioria das quais não corresponde a nada no UNIX.

UNIX	Win32	Descrição
fork	CreateProcess	Cria um novo processo
waitpid	WaitForSingleObject	Pode esperar que um processo termine
execve	(nenhuma)	CreateProcess = fork + execve
exit	ExitProcess	Conclui a execução
open	CreateFile	Cria um arquivo ou abre um arquivo existente
close	CloseHandle	Fecha um arquivo
read	ReadFile	Lê dados a partir de um arquivo
write	WriteFile	Escreve dados em um arquivo
lseek	SetFilePointer	Move o ponteiro do arquivo
stat	GetFileAttributesEx	Obtém vários atributos do arquivo
mkdir	CreateDirectory	Cria um novo diretório
rmdir	RemoveDirectory	Remove um diretório vazio
link	(nenhuma)	Win32 não dá suporte a ligações
unlink	DeleteFile	Destrói um arquivo existente
mount	(nenhuma)	Win32 não dá suporte a mount
umount	(nenhuma)	Win32 não dá suporte a mount
chdir	SetCurrentDirectory	Altera o diretório de trabalho atual
chmod	(nenhuma)	Win32 não dá suporte a segurança (embora o NT suporte)
kill	(nenhuma)	Win32 não dá suporte a sinais
time	GetLocalTime	Obtém o tempo atual

Chamadas de sistema: No código.

```
/* Abre o arquivo de entrada e cria o arquivo de saída*/
in_fd = open(argv[1], O_RDONLY);          /* abre o arquivo de origem */
if (in_fd < 0) exit(2);                    /* se não puder ser aberto, saia */
out_fd = creat(argv[2], OUTPUT_MODE);     /* cria o arquivo de destino */
if (out_fd < 0) exit(3);                   /* se não puder ser criado, saia */

/* Laco de copia */
while (TRUE) {
    rd_count = read(in_fd, buffer, BUF_SIZE); /* le um bloco de dados */
    if (rd_count <= 0) break                  /* se fim de arquivo ou erro, sai do laco */
    wt_count = write(out_fd, buffer, rd_count); /* escreve dados */
    if (wt_count <= 0) exit(4);               /* wt_count <= 0 e um erro */
}

/* Fecha os arquivos */
close(in_fd);
close(out_fd);
```

Chamadas de sistema: No código.

```

/* Abre o arquivo de entrada e cria o arquivo de saída*/
in_fd = open(argv[1], O_RDONLY);          /* abre o arquivo de origem */
if (in_fd < 0) exit(2);                    /* se não puder ser aberto, saia */
out_fd = creat(argv[2], OUTPUT_MODE);     /* cria o arquivo de destino */
if (out_fd < 0) exit(3);                   /* se não puder ser criado, saia */

/* Laco de copia */
while (TRUE) {
    rd_count = read(in_fd, buffer, BUF_SIZE); /* le um bloco de dados */
    if (rd_count <= 0) break                  /* se fim de arquivo ou erro, sai do laco */
    wt_count = write(out_fd, buffer, rd_count); /* escreve dados */
    if (wt_count <= 0) exit(4);               /* wt_count <= 0 e um erro */
}

/* Fecha os arquivos */
close(in_fd);
close(out_fd);

```

Se cada SO possui diferentes chamadas de sistema, como o mesmo código pode funcionar em diferentes sistemas?

Chamadas de sistema: No código.

```
/* Abre o arquivo de entrada e cria o arquivo de saída*/
in_fd = open(argv[1], O_RDONLY);          /* abre o arquivo de origem */
if (in_fd < 0) exit(2);                    /* se não puder ser aberto, saia */
out_fd = creat(argv[2], OUTPUT_MODE);     /* cria o arquivo de destino */
if (out_fd < 0) exit(3);                   /* se não puder ser criado, saia */

/* Laco de copia */
while (TRUE) {
    rd_count = read(in_fd, buffer, BUF_SIZE); /* le um bloco de dados */
    if (rd_count <= 0) break                  /* se fim de arquivo ou erro, sai do laco */
    wt_count = write(out_fd, buffer, rd_count); /* escreve dados */
    if (wt_count <= 0) exit(4);               /* wt_count <= 0 é um erro */
}

/* Fecha os arquivos */
close(in_fd);
close(out_fd);
```

O compilador converte a chamada da linguagem de programação, para a chamada do sistema operacional alvo

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Processos

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Conceitualização

Processos

O conceito mais central em qualquer sistema operacional é o processo:

- **Processo é uma abstração de um programa em execução.**
- Sem a abstração de processo, a computação moderna não poderia existir!
- São uma das mais antigas e importantes abstrações em SO.
- **Processos dão suporte à possibilidade de haver operações concomitantes, mesmo quando há apenas uma CPU disponível, transformando uma única CPU em múltiplas CPUs virtuais.**

Com relação à multiprogramação, processo essencialmente é um contêiner que armazena todas as informações necessárias para executar um programa.

Conceitualização

Processos

A diferença entre um processo e um programa é sutil, mas absolutamente crucial.

Uma analogia para ajudar a compreender:

Considere que você gosta de cozinhar e está preparando um bolo de aniversário para sua filha mais nova...

- Você tem uma receita de um bolo de aniversário
- Você tem uma cozinha bem estocada com todos os ingredientes

Conceitualização

Processos

Nessa analogia:

- A receita é o programa;
- Você é o processador (CPU);
- E, os ingredientes do bolo são os dados de entrada.

O processo é a atividade consistindo na leitura da receita, busca de ingredientes e preparo do bolo por você.

Conceitualização



Processos

Ainda nessa analogia... Agora imagine que seu filho aparece correndo chorando, dizendo que foi picado por uma abelha. **[INTERRUPÇÃO]**

Você registra onde ele estava na receita (o estado do processo atual é salvo), pega um livro de primeiros socorros e começa a seguir as orientações. **[MULTIPROGRAMAÇÃO]**

→ Isto é o processador sendo trocado de um processo (preparo do bolo) para um processo mais prioritário (prestar cuidado médico), cada um tendo um programa diferente (receita X livro de primeiros socorros).

→ Quando a picada de abelha tiver sido cuidada, você volta para o seu bolo, continuando do ponto onde ele havia parado.

Contexto

Multiprogramação

Considerando um **sistema computacional com uma única CPU**, sabe-se que num **determinado instante a CPU está executando apenas um processo**. Em **sistemas de multiprogramação**, a CPU muda de um processo para outro **rapidamente**, executando cada um por dezenas ou centenas de milissegundos.

→ Ou seja, **em apenas 1s, pode-se trabalhar em vários processos**. Essa é a ilusão do paralelismo, também chamada de **(pseudo) concorrência**.

Para **chips multinúcleos**, contamos cada núcleo como uma CPU.

→ Portanto, quando diz-se que uma CPU pode executar apenas um processo de cada vez, **considere que cada núcleo pode executar apenas um processo de cada vez**.

O modelo de processo

Controlar múltiplas atividades em paralelo é algo difícil para as pessoas realizarem. Portanto, **projetistas de sistemas operacionais** através dos anos **desenvolveram um modelo conceitual (processos sequenciais) que torna o paralelismo algo mais fácil de lidar**. Nesse modelo, todos os softwares executáveis no computador, incluindo o sistema operacional, são organizados em uma série de processos sequenciais.

→ **Conceitualmente, cada processo tem sua própria CPU virtual.**

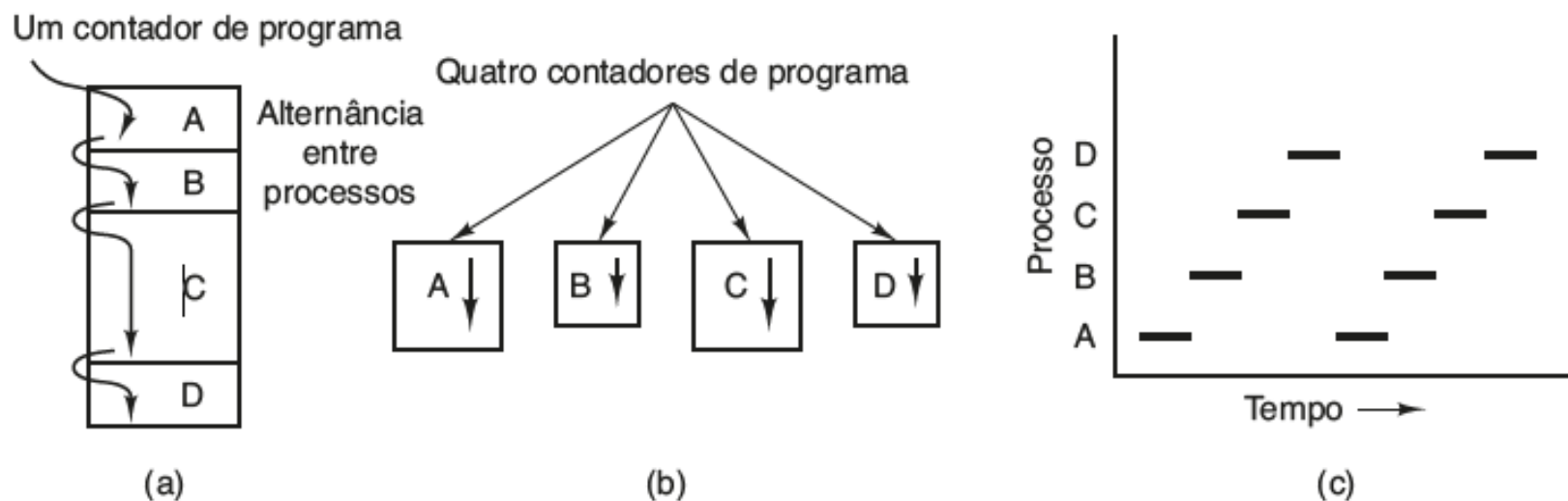
A multiprogramação é a troca realizada a todo momento de processo em processo para ser executado na CPU real.

O modelo de processo

Processos sequenciais

- Se cada processo tem sua própria CPU virtual... Então:
- Cada um tem o seu PC (*Program Counter* – contador de programa).

FIGURA 2.1 (a) Multiprogramação de quatro programas. (b) Modelo conceitual de quatro processos sequenciais independentes. (c) Apenas um programa está ativo de cada vez.



Ciclo de vida de processos

Criação de processos

Resumidamente, um novo processo é criado por outro já existente executando uma chamada de sistema de criação de processo.

Esse outro processo pode ser um processo de usuário sendo executado, um processo de sistema invocado do teclado ou mouse, ou um processo gerenciador de lotes.

O que esse processo faz é executar uma chamada de sistema para criar o novo processo. Essa chamada de sistema diz ao sistema operacional para criar um novo processo e indica, direta ou indiretamente, qual programa executar nele.

Ciclo de vida de processos

Estados de processos

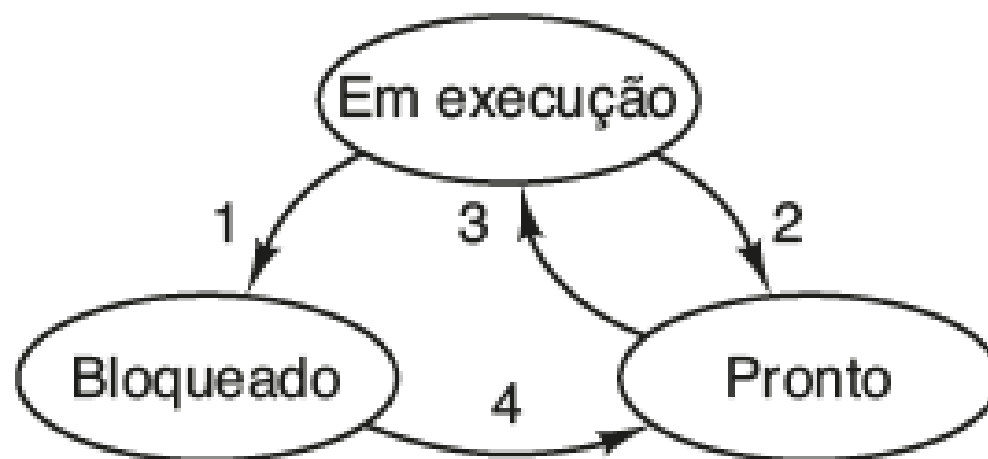
A Figura 2.2 (a seguir) apresenta um diagrama de estado, mostrando os três estados nos quais um processo pode se encontrar:

1. **Em execução** (realmente usando a CPU naquele instante).
2. **Pronto** (executável, temporariamente parado para deixar outro processo ser executado).
3. **Bloqueado** (incapaz de ser executado até que algum evento externo aconteça).

Ciclo de vida de processos

Estados de processos

FIGURA 2.2 Um processo pode estar nos estados em execução, bloqueado ou pronto. Transições entre esses estados ocorrem como mostrado.



1. O processo é bloqueado aguardando uma entrada
2. O escalonador seleciona outro processo
3. O escalonador seleciona esse processo
4. A entrada torna-se disponível

Ciclo de vida de processos

Criação de processos

Quatro eventos principais fazem com que os processos sejam criados:

1. Inicialização do sistema.

Quando um sistema operacional é inicializado, em geral uma série de processos é criada.

→ Processos de primeiro plano, são processos que interagem com usuários humanos.

→ Processos que executam no segundo plano (*background*), por exemplo o servidor de e-mails ou o servidor *web*, são chamados de *daemons* ou serviços.

Ciclo de vida de processos

Criação de processos

Quatro eventos principais fazem com que os processos sejam criados:

2. Execução de uma chamada de sistema de criação.

Um processo em execução pode emitir chamadas de sistema para criar um ou mais processos novos para ajudá-lo em seu trabalho.

→ Muito útil quando o trabalho pode ser facilmente formulado em termos de vários de processo, controlados por um processo em execução.

Por exemplo, se uma grande quantidade de dados está sendo online para processamento subsequente, pode ser conveniente criar um processo para buscar os dados e colocá-los em um local compartilhado de memória enquanto um segundo processo remove os itens de dados e os processa.

Ciclo de vida de processos

Criação de processos

Quatro eventos principais fazem com que os processos sejam criados:

3. Solicitação de um usuário para criar um novo processo.

Em **sistemas interativos**, os usuários podem **começar um programa digitando um comando ou clicando duas vezes sobre um ícone**.

→ Cada uma dessas ações inicia um novo processo e executa nele o programa selecionado.

Ciclo de vida de processos

Criação de processos

Quatro eventos principais fazem com que os processos sejam criados:

4. Início de uma tarefa em lote.

A última situação na qual processos são criados aplica-se somente aos sistemas em lote encontrados em grandes computadores.

Por exemplo: Pense no gerenciamento de estoque ao fim de um dia em uma cadeia de lojas, nesse caso **usuários podem submeter tarefas em lote ao sistema** (possivelmente de maneira remota).

→ **Quando o sistema operacional decide que ele tem os recursos para executar outra tarefa, ele cria um novo processo e executa a próxima tarefa a partir da fila de entrada nele.**

Ciclo de vida de processos

Término de processos

Após um processo ter sido criado, ele começa a ser executado e realiza qualquer que seja o seu trabalho. No entanto, nada dura para sempre, nem mesmo os processos.

Ciclo de vida de processos

Término de processos

O processo terminará, normalmente devido a uma das condições a seguir:

1. Saída normal (voluntária).

→ A maioria dos processos termina por terem realizado o seu trabalho.

Ex: Compilador termina de traduzir o programa; Ou, programas baseados em tela, que também dão suporte ao término voluntário.

Ciclo de vida de processos

Término de processos

O processo terminará, normalmente devido a uma das condições a seguir:

2. Erro fatal (involuntário).

→ A segunda razão para o término é a que o processo descobre um erro fatal.

Ex: Compilador chamado para compilar arquivo inexistente.

→ *gcc meu código-fonte.c*

Processos interativos, com base em tela, geralmente não fecham quando parâmetros ruins são dados. Normalmente apresentam uma caixa de diálogo e pedem ao usuário para tentar de novo.

Ciclo de vida de processos

Término de processos

O processo terminará, normalmente devido a uma das condições a seguir:

3. Saída por erro (voluntária).

→ A terceira razão para o término é um erro causado pelo processo, muitas vezes decorrente de um erro de programa.

Exemplos incluem executar uma instrução ilegal, referenciar uma memória não existente, ou dividir por zero.

→ Tratamento de exceções (*exceptions*) e erros são importantes!

Ciclo de vida de processos

Término de processos

O processo terminará, normalmente devido a uma das condições a seguir:

4. Morto por outro processo (involuntário).

→ A quarta razão pela qual um processo pode ser finalizado ocorre quando o processo executa uma chamada de sistema dizendo ao sistema operacional para matar outro processo.

Em UNIX, essa chamada é *kill*.

Em Win32 a chamada correspondente é *TerminateProcess*.

Execução, componentes e características do processo

Execução

- Primeiro plano
- Segundo plano

Componentes

- Instruções
- Espaço de endereçamento
- Contextos de hardware e software

Características

- I/O-bound process
- CPU bound process

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Threads

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

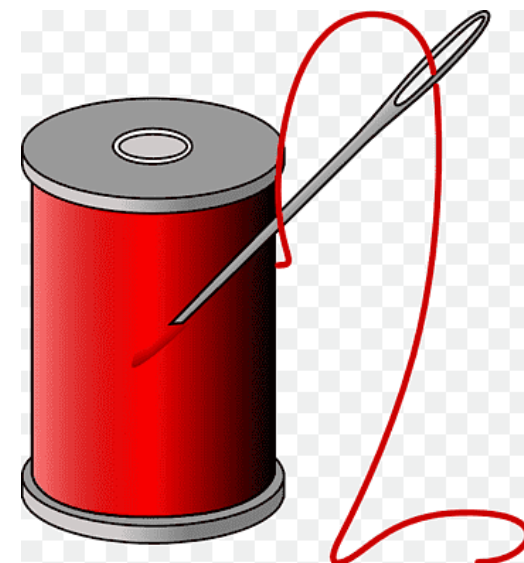
Conceitualização e contexto

Processos

→ Processo visto como um modo para agrupar recursos relacionados.

Outro conceito que um processo tem é de uma **linha (*thread*) de execução**, normalmente abreviado para apenas *thread*.

A *thread* tem um contador de programa que controla qual instrução deve ser executada em seguida.



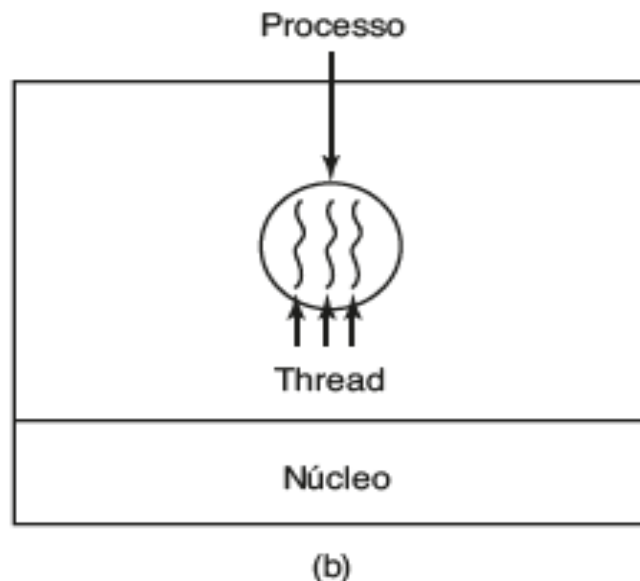
Conceitualização e contexto

Processos

→ Processo visto como um modo para agrupar recursos relacionados.

Contudo, em muitas situações deseja-se ter **múltiplas linhas de execução no mesmo espaço de endereçamento**, executando em quase paralelo, como se fossem processos separados.

FIGURA 2.11 (a) Três processos, cada um com um thread. (b) Um processo com três threads.



Conceitualização e contexto

Threads

A *thread* é a parte do processo que controla a execução do processo.

- **Processos são usados para agrupar recursos;**
- ***Threads* são as entidades escalonadas para execução na CPU.**
 - É a ***thread*** que possui o contador de programa que controla qual instrução deve ser executada em seguida.
 - Por terem algumas das propriedades dos processos, às vezes ***threads*** são chamadas de miniprocessos ou processos leves.

Modelo de *threads*

Threads

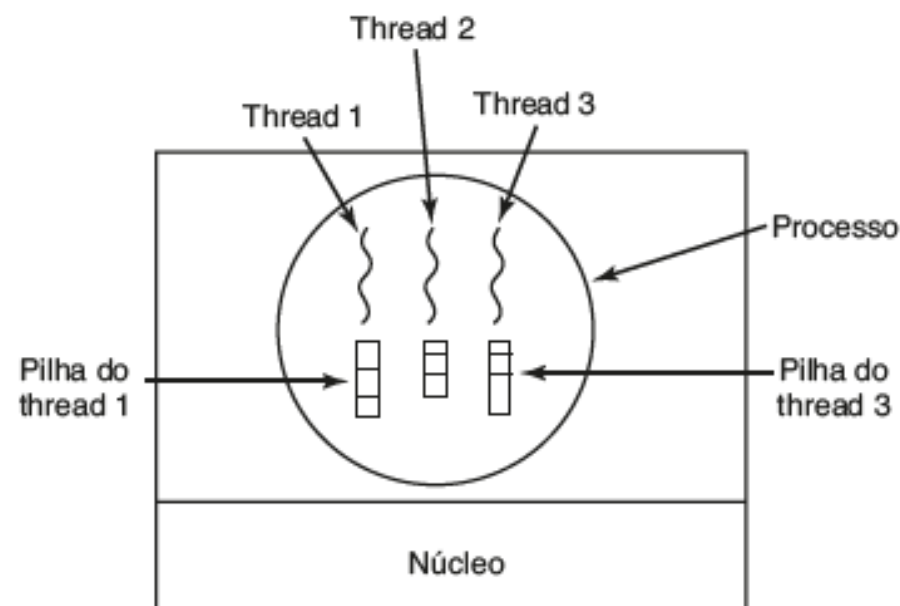
A *thread* é a parte do processo que controla a execução do processo.

- **Processos são usados para agrupar recursos;**
- ***Threads* são as entidades escalonadas para execução na CPU.**

FIGURA 2.12 A primeira coluna lista alguns itens compartilhados por todos os threads em um processo. A segunda lista alguns itens específicos a cada thread.

Itens por processo	Itens por thread
Espaço de endereçamento	Contador de programa
Variáveis globais	Registradores
Arquivos abertos	Pilha
Processos filhos	Estado
Alarmes pendentes	
Sinais e tratadores de sinais	
Informação de contabilidade	

FIGURA 2.13 Cada thread tem a sua própria pilha.



Utilização de *threads*

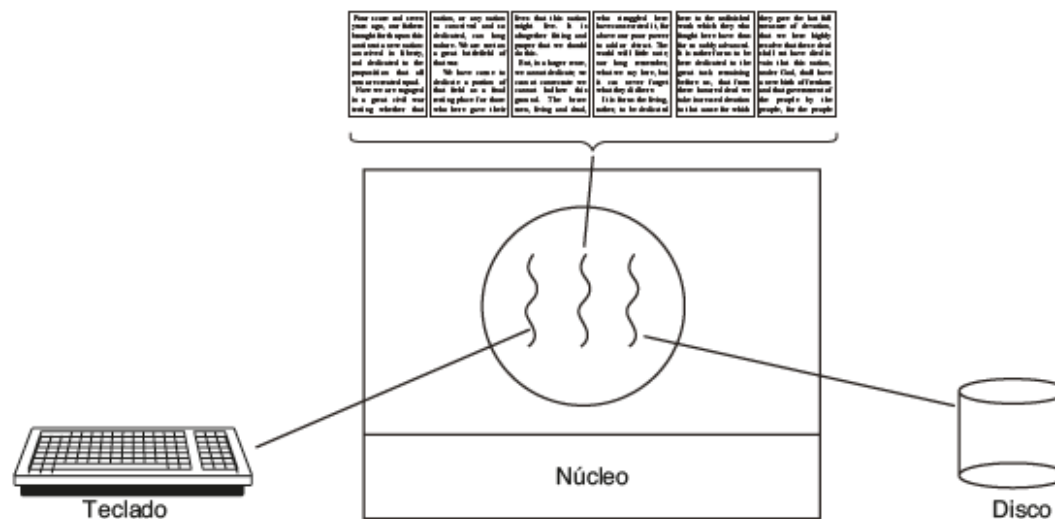
Por que alguém iria querer ter um tipo de processo dentro de um processo?

- A principal razão para o uso de *threads* é o modelo de **programação** que permite **múltiplas** *threads* sequenciais serem executadas quase paralelamente.
 - **Possibilidade de ter diversas linhas de execução ao mesmo tempo.**
- Um segundo argumento para a existência dos *threads* é o custo computacional, pois elas são mais leves do que os processos. *Threads* são mais fáceis e rápidos para criar e destruir do que os processos.
 - **Criar *thread* é de 10 a 100 vezes mais rápido do que criar um processo.**
- Também são úteis em sistemas com **múltiplas CPUs** com **paralelismo real**.

Utilização de *threads*

Exemplo I) Editor de textos

FIGURA 2.7 Um processador de texto com três threads.



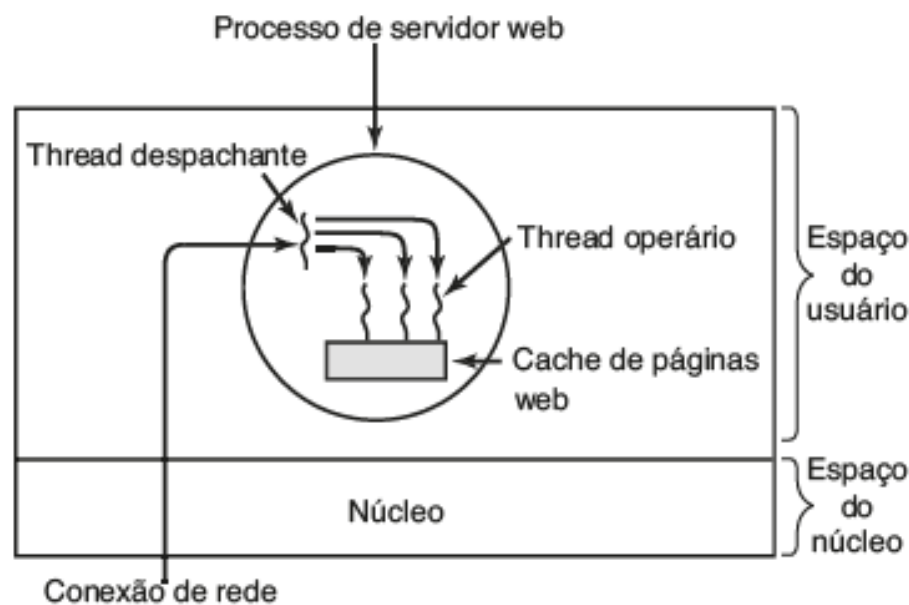
Uma *thread* interage com o usuário, outra lida com a formatação em segundo plano, e terceira é responsável por salvar automaticamente o documento no disco.

→ Se fosse *monothread*, o *backup* bloquearia interatividade.

Utilização de *threads*

Exemplo II) Servidor web *multithread*

FIGURA 2.8 Um servidor web multithread.



Para cada conexão recebida, ou seja, para cada página solicitada, o servidor recebe a requisição por uma *thread* despachante, a qual inicia *threads* para o processamento.

Utilização de *threads*

Exemplo II) Servidor web *multithread*

Sem *threads*, seriam quase
20 minutos de espera!

1.140 usuários simultâneos

Se cada requisição levar 1s:

$1140/60 = 19$ minutos

it	Sessions						In
	Cur	Max	Limit	Total	LbTot	Last	
-	1 140	1 694	20 000	51 914			380 962 77

rate	Sessions						
Limit	Cur	Max	Limit	Total	LbTot	Last	In
	133 *****	209	-	55 811 *****	4 988	1s	73 724 9
	175 *****	223	-	39 261 *****	4 970	1s	57 249 8
	190 *****	301	-	43 282 *****	4 971	2s	61 928 8
	157 *****	213	-	39 795 *****	4 994	0s	58 067 0
	113 *****	280	-	41 854 *****	4 993	0s	64 200 4
	202 *****	265	-	35 773 *****	4 984	0s	57 337 9
	0 ***	0	-	0 ***	0	?	
	0 ***	0	-	0 ***	0	?	
	0 ***	0	-	0 ***	0	?	
	0 ***	0	-	0 ***	0	?	
	970	1 429	2 000	255 784 *****	29 900	0s	372 547 3

Threads POSIX



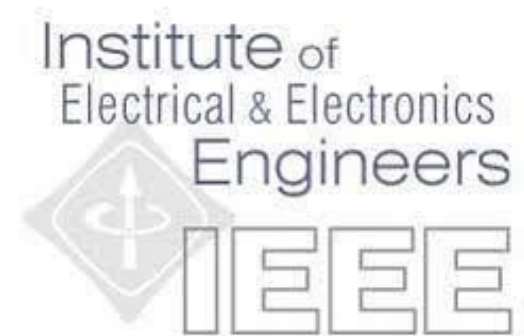
Padrão IEEE 1003.1c – Pacote de *threads* chamado *Pthreads*

→ O padrão define mais de 60 chamadas de função (*system calls*).

Todas as *threads* têm determinadas propriedades.

Cada uma tem um **identificador**, um conjunto de **registradores (incluindo o contador de programa)**, e um **conjunto de atributos**, que são armazenados em uma estrutura.

Os atributos incluem **tamanho da pilha**, **parâmetros de escalonamento** e **outros itens necessários** para usar *threads*.



Threads POSIX



Padrão IEEE 1003.1c – Pacote de *threads* chamado *Pthreads*

FIGURA 2.14 Algumas das chamadas de função do Pthreads.

Chamada de thread	Descrição
Pthread_create	Cria um novo thread
Pthread_exit	Conclui a chamada de thread
Pthread_join	Espera que um thread específico seja abandonado
Pthread_yield	Libera a CPU para que outro thread seja executado
Pthread_attr_init	Cria e inicializa uma estrutura de atributos do thread
Pthread_attr_destroy	Remove uma estrutura de atributos do thread

Sincronização, Tipos e Escalonamento

Sincronização de *threads*

1. *Join*
2. *Sleep*

Tipos e Escalonamento de *threads*

1. *Threads* no Modo Usuário
2. *Threads* no Modo Núcleo
3. *Threads* Híbridas

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Comunicação entre Processos

Interprocess Communication - IPC

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

Introdução

Comunicação entre processos – Questões fundamentais:

- **Como um processo pode passar informações para outro?**

Ex: pipeline do interpretador de comandos, onde a saída do primeiro processo tem de ser passada para o segundo. Como visto no slide anterior.

- **Como certificar-se de que dois ou mais processos não se atrapalhem?**

Ex: dois processos em um sistema de reserva de voos tentando ficar com o último assento em um avião, cada um com um cliente diferente interessado.

- **E quanto ao sequenciamento adequado quando há dependências?**

Ex: Se o processo A produz dados e o processo B os imprime, B tem de esperar até que A tenha produzido alguns dados antes de começar a imprimir.

→ **É disso que trata a Comunicação entre Processos** ←

Introdução

Comunicação entre *threads* – Questões fundamentais:

- **Como uma *thread* pode passar informações para outra?**
 - Com *threads*, passar informações é fácil, pois elas compartilham um espaço de endereçamento.
- **Como certificar-se de que duas ou mais *threads* não se atrapalhem?**

Ex: duas *threads* do sistema de reserva de voos tentando ficar com o último assento em um avião, cada uma com um cliente diferente interessado.
- **E quanto ao sequenciamento adequado quando há dependências?**

Ex: Se a *thread* A produz dados e a *thread* B os imprime, B tem de esperar até que A tenha produzido alguns dados antes de começar a imprimir.

→ **Os mesmos problemas e soluções se aplicam às *threads*** ←

Condição de corrida e Região crítica

Condição de corrida (*race condition*)

Situações em que dois ou mais processos estão lendo ou escrevendo alguns dados compartilhados, de modo que o resultado final depende precisamente de quem e quando executa.

Exemplo: Fila de impressão

in = local livre que indica a entrada da fila

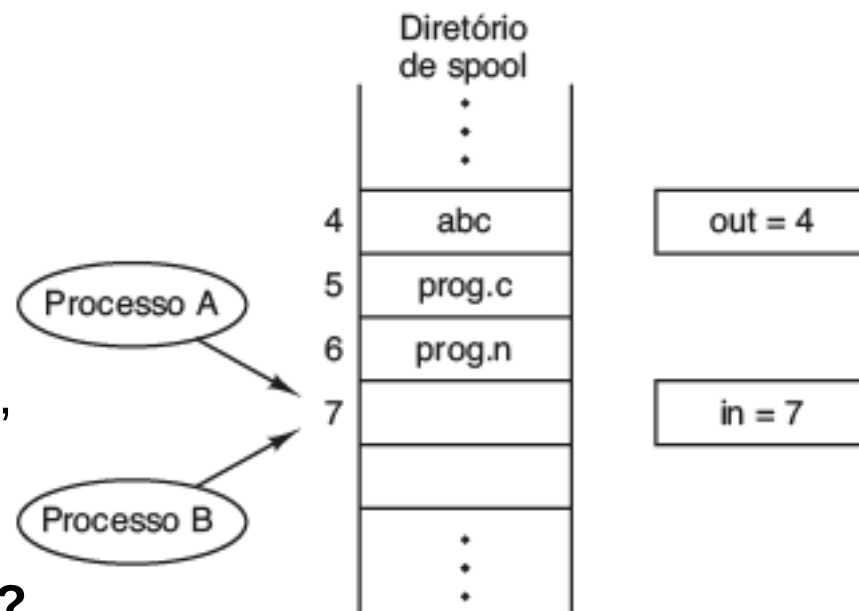
out = próximo arquivo a ser impresso

A e B precisam enviar documentos.

Começam verificando o endereço de entrada, alocam o endereço 7 e incrementam *in*.

E se sofreram parada neste procedimento?

FIGURA 2.21 Dois processos querem acessar a memória compartilhada ao mesmo tempo.



Condição de corrida e Região crítica

Região Crítica

É chamada de região crítica ou seção crítica, a parte do programa onde a memória compartilhada é manipulada. Exemplo: Fila de impressão.

```
/* Variáveis compartilhadas */
Document *printer_queue[100];
int in = 0;

/* Função de alocação de documento na fila de impressão */
void* send_job_to_printer(void* arg) {
    /* INÍCIO DA REGIÃO CRÍTICA */
    int my_slot = in; // Obtendo valor do slot de entrada
    >>> POSSÍVEL PONTO DE PARADA POR ESCALONADOR PREEMPTIVO <<<
    printer_queue[my_slot] = (Document*) arg; // Alocando documento
    in++; // Incrementando a variável compartilhada
    /* FIM DA REGIÃO CRÍTICA */
}
```

Condição de corrida e Região crítica

Como evitar as condições de corrida?

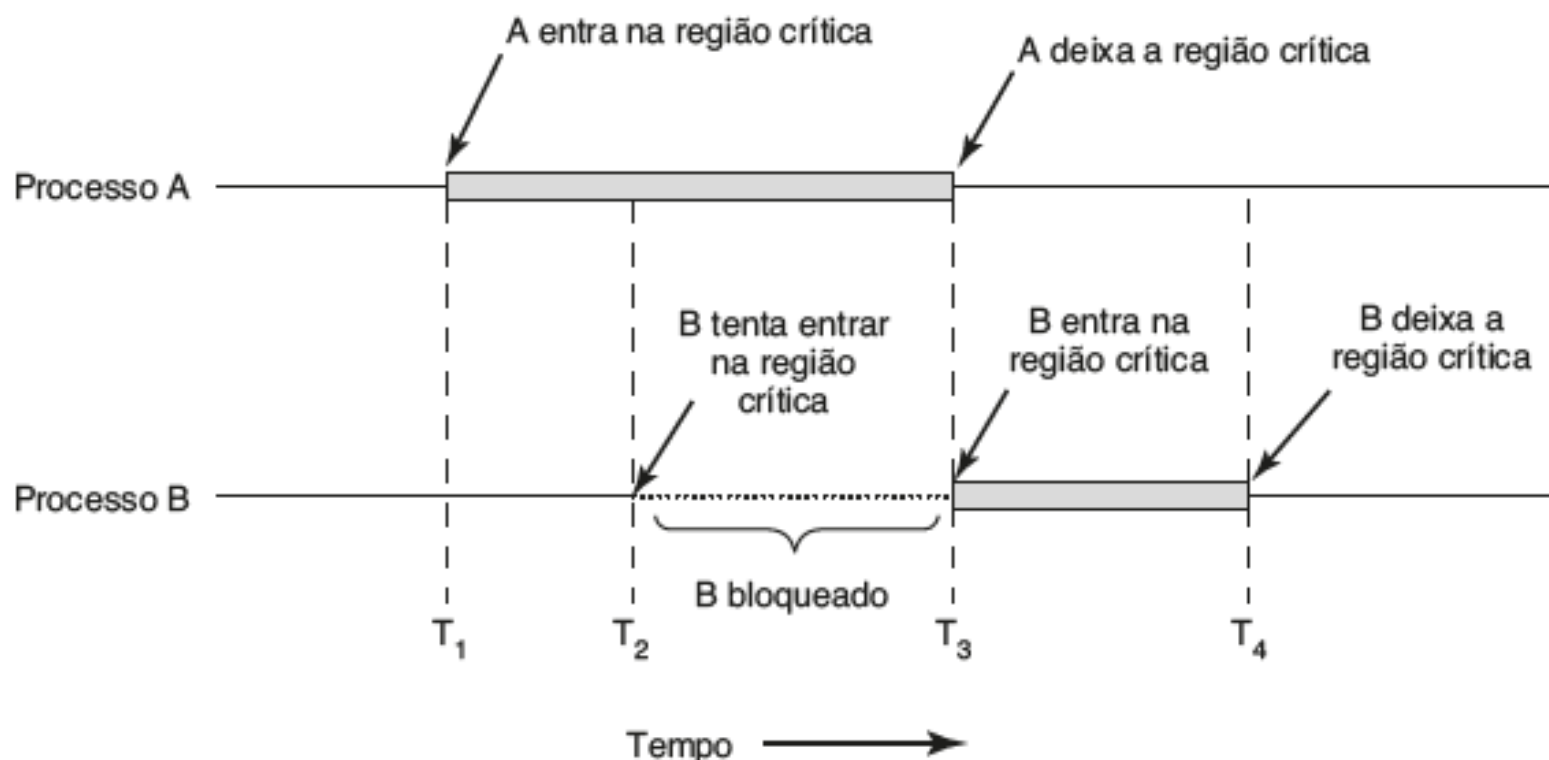
A chave para evitar problemas envolvendo algo compartilhado (memória, arquivos, e tudo o mais compartilhado) é encontrar alguma maneira de proibir mais de um processo de ler e escrever os dados compartilhados ao mesmo tempo.

Condição de corrida e Região crítica

Como evitar as condições de corrida?

Isto é, **realizar a exclusão mútua**, se um processo está usando um arquivo ou variável compartilhada, os outros serão impedidos de realizar o mesmo.

FIGURA 2.22 Exclusão mútua usando regiões críticas.



Condição de corrida e Região crítica

Como evitar as condições de corrida?

Organizar as coisas de maneira que jamais dois processos estejam em suas regiões críticas ao mesmo tempo pode evitar as corridas. Apesar desta exigência evitar condições de corrida, ela não é suficiente para garantir processos paralelos cooperando correta e eficientemente no uso de dados compartilhados.

Requisitos necessários para ter-se uma boa solução de exclusão mútua:

1. Dois processos jamais simultaneamente em suas regiões críticas.
2. Nenhuma suposição a respeito de velocidades ou do número de CPUs.
3. Nenhum processo fora de sua região crítica pode bloquear processos.
4. Nenhum processo deve esperar eternamente para executar região crítica.

...

Abordagens para realização da Exclusão Mútua

...

Exclusão mútua com espera ocupada

Desabilitando interrupções

Com as interrupções desabilitadas, nenhuma interrupção de relógio poderá ocorrer. Com as interrupções desligadas, a CPU não será chaveada para outro processo. Assim que um processo tiver desabilitado as interrupções, ele poderá examinar e atualizar a memória compartilhada sem medo de que qualquer outro processo interfira.

→ **É imprudente permitir processos de usuário desligar interrupções!**

E se um deles desligasse uma interrupção e nunca mais a ligasse de volta?

→ **Fere requisito 2, pois supõe que a arquitetura é de CPU única.**

Desabilitar interrupções afeta somente a CPU que executou comando *disable*.

Exclusão mútua com espera ocupada

Variáveis do tipo trava

Como segunda tentativa, vamos procurar por uma solução de *software*:

Considere ter uma única variável (de trava) compartilhada. Quando um processo quer entrar em sua região crítica, ele primeiro testa a trava. Se a trava é 0, o processo a configura para 1 e entra na região crítica. Se a trava já é 1, o processo apenas espera até que ela se torne 0.

→ **Essa ideia tem a mesma falha fatal do exemplo da Fila de Impressão!**

O processo pode ser escalonado antes que ele possa configurar a trava para 1.

→ **Fere requisito 1, permite processos simultaneamente em região crítica.**

Exclusão mútua com espera ocupada

Alternância explícita

Testa continuamente uma variável até que algum valor apareça.

Dois ou mais processos constantemente verificam o valor da variável *turn* para determinar se o processo poderá entrar em sua região crítica. Após a execução da região crítica, o processo altera o valor de *turn* para liberar outro processo para execução de região crítica.

FIGURA 2.23 Uma solução proposta para o problema da região crítica. (a) Processo 0. (b) Processo 1.
Em ambos os casos, certifique-se de observar os pontos e vírgulas concluindo os comandos while.

```
while (TRUE) {  
    while (turn !=0)          /* laço */;  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

```
while (TRUE) {  
    while (turn !=1)          /* laço */;  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

→ Fere requisito 3, bloqueia processos quando não está na região crítica.

Exclusão mútua com espera ocupada

Solução de Peterson

Combina a ideia de alternar a vez, com a ideia das variáveis de controle.

FIGURA 2.24 A solução de Peterson para realizar a exclusão mútua.

```

#define FALSE 0
#define TRUE 1
#define N      2                                /* numero de processos */

int turn;                                       /* de quem e a vez? */
int interested[N];                             /* todos os valores 0 (FALSE) */

enter_region void enter_region(int process);    /* processo e 0 ou 1 */
{
    int other;                                  /* numero do outro processo */

    other = 1 - process;                        /* o oposto do processo */
    interested[process] = TRUE;                 /* mostra que voce esta interessado */
    turn = process;                             /* altera o valor de turn */
    while (turn == process && interested[other] == TRUE) /* comando nulo */ ;
}

leave_region void leave_region(int process)    /* processo: quem esta saindo */
{
    interested[process] = FALSE;                /* indica a saida da regioao critica */
}

```

Exclusão mútua com espera ocupada

A instrução TSL

Solução baseada em hardware.

Alguns computadores possuem uma **instrução** como **TSL RX,LOCK** (*Test and Set Lock* — teste e configure trava) que **fornece uma operação atômica para leitura e atualização da variável lock, de acordo com o registrador RX**. A CPU executando a instrução TSL impede o acesso ao barramento de memória para proibir que outras CPUs acessem a memória até ela terminar.

FIGURA 2.25 Entrando e saindo de uma região crítica usando a instrução TSL.

enter_region:	
TSL REGISTER,LOCK	I copia lock para o registrador e poe lock em 1
CMP REGISTER,#0	I lock valia zero?
JNE enter_region	I se fosse diferente de zero, lock estaria ligado; portanto, continue no laço de repeticao
RET	I retorna a quem chamou; entrou na regioao critica
leave_region:	
MOVE LOCK,#0	I coloque 0 em lock
RET	I retorna a quem chamou

Exclusão mútua com espera ocupada

A instrução XCHG

Outra solução baseada em hardware.

A instrução XCHG troca os conteúdos de duas posições atômicamente. Por exemplo, um registrador e uma palavra em memória. **É essencialmente o mesmo que a solução com TSL.**

CPUs Intel x86 usam a instrução XCHG para a sincronização de baixo nível.

FIGURA 2.26 Entrando e saindo de uma região crítica usando a instrução XCHG.

enter_region:	
MOVE REGISTER,#1	insira 1 no registrador
XCHG REGISTER,LOCK	substitua os conteudos do registrador e a variacao de lock
CMP REGISTER,#0	lock valia zero?
JNE enter_region	se fosse diferente de zero, lock estaria ligado; portanto
	continue no laco de repeticao
RET	retorna a quem chamou; entrou na regioao critica
leave_region:	
MOVE LOCK,#0	coloque 0 em lock
RET	retorna a quem chamou

Exclusão mútua com espera ocupada

Considerações sobre exclusão mútua com espera ocupada

A solução de Peterson e as soluções com TSL ou XCHG, satisfazem os requisitos de boas soluções.

Entretanto, as soluções apresentadas possuem o **defeito crônico da espera ocupada: Há desperdício da CPU na contínua verificação do enquanto o processo está em espera.**

Na essência, o que essas soluções fazem é o seguinte: quando um processo quer entrar em sua região crítica, ele confere para ver se a entrada é permitida. Se não for, o processo apenas esperará em um laço apertado até que isso seja permitido. *É por isto que se chama espera ocupada, pois a CPU fica completamente ocupada com as verificações até que seja permitido o processo entrar em sua região crítica.*

Outras abordagens para exclusão mútua

Dormir e acordar

Uso de primitivas de comunicação entre processos que bloqueiam, ao invés de desperdiçar tempo da CPU quando eles não são autorizados a entrar nas suas regiões críticas.

Sleep: chamada de sistema que faz o processo ser bloqueado. Isto é, seja suspenso até que outro processo o desperte.

Wakeup: chamada de sistema responsável por despertar (desbloquear) um processo informado por parâmetro.

Outras abordagens para exclusão mútua

Semáforo (proposto por E. W. Dijkstra em 1965)

A operação *down* em um semáforo **verifica se o valor é maior do que 0. Se for, ele decrementará o valor** (isto é, gasta um sinal de acordar armazenado) e apenas continua. **Se o valor for 0, o processo é colocado para dormir *sem completar o down*** para o momento.

A operação *up* incrementa o valor de um determinado semáforo. Se um ou mais processos estiverem dormindo naquele semáforo, incapaz de completar uma operação *down* anterior, um deles é escolhido pelo sistema e é autorizado a completar seu *down*. Desse modo, após um *up* com processos dormindo em um semáforo, ele ainda estará em 0, mas haverá menos processos dormindo nele.

As operação de semáforo são indivisíveis. Essa atomicidade é essencial para solucionar problemas de sincronização e evitar condições de corrida.

Outras abordagens para exclusão mútua

Mutex

Duas rotinas são usadas com mutexes.

A execução de ***mutex_lock*** verificará se o mutex está destravado naquele momento, **significando que a região crítica está disponível**. Então, a chamada seguirá e a *thread* que chamou estará livre para entrar na região crítica.

Se o mutex já estiver travado, a *thread* que chamou será bloqueada até que a *thread* na região crítica tenha concluído e chame ***mutex_unlock***.

Se múltiplas *threads* estiverem bloqueadas no mutex, uma delas será escolhida ao acaso e liberado para adquirir a trava.

Mutexes são muito simples, podem ser implementados facilmente no espaço do usuário, quando instruções TSL ou XCHG estiverem disponíveis.

Outras abordagens para exclusão mútua

Considerações sobre Semáforos e Mutexes

Semáforos e mutexes tornam a comunicação entre processos fácil, certo?

Errado!

Observe de perto a ordem dos *downs* antes de inserir ou remover itens do *buffer* na Figura 2.28. Suponha que os dois *downs* no código do produtor fossem invertidos, de maneira que mutex tenha sido decrescido antes de *empty* em vez de depois dele.

Se o *buffer* estivesse completamente cheio, o produtor seria bloqueado, com mutex configurado para 0. Em consequência, da próxima vez que o consumidor tentasse acessar o *buffer*, ele faria um down em mutex, agora 0, e seria bloqueado também. **Ambos os processos ficariam bloqueados para sempre e nenhum trabalho seria mais realizado.**

Outras abordagens para exclusão mútua

Considerações sobre Semáforos e Mutexes

FIGURA 2.28 O problema do produtor-consumidor usando semáforos.

```

#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

/* numero de lugares no buffer */
/* semaforos sao um tipo especial de int */
/* controla o acesso a regio critica */
/* conta os lugares vazios no buffer */
/* conta os lugares preenchidos no buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item( );
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item( );
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}

```

Outras abordagens para exclusão mútua

Considerações sobre Semáforos e Mutexes

Semáforos e mutexes tornam a comunicação entre processos fácil, certo?

Errado!

Esse problema é destacado para mostrar o quão cuidadoso você precisa ser quando usa semáforos.

→ Um erro sutil e tudo para completamente.

É como programar em linguagem de montagem (*assembly*), mas pior.

→ Em IPC, erros são condições de corrida, impasses e outras formas de comportamento imprevisível e irreproduzível.

Outras abordagens para exclusão mútua

Monitores (propostos por Brinch Hansen, 1973; e, Hoare, 1974)

Monitores são uma construção da linguagem de programação, então o compilador sabe que eles são especiais e podem lidar com chamadas para rotinas de monitor diferentemente de outras chamadas. A Figura 2.33 ilustra um monitor escrito em uma linguagem imaginária, Pidgin Pascal.

FIGURA 2.33 Um monitor.

```
monitor example
  integer i;
  condition c;

  procedure producer ( );
  .
  .
  .
  end;

  procedure consumer ( );
  .
  .
  .
  end;
end monitor;
```

→ **Monitor é um conceito que a linguagem C não tem!** ←

Outras abordagens para exclusão mútua

Monitores (propostos por Brinch Hansen, 1973; e, Hoare, 1974)

FIGURA 2-35 Uma solução para o problema produtor-consumidor

```

public class ProducerConsumer {
    static final int N = 100; // tamanho do buffer
    static producer p = new producer();
    static consumer c = new consumer();
    static our_monitor mon = new our_monitor();

    public static void main(String args[]) {
        p.start(); // inicia o thread produtor
        c.start(); // inicia o thread consumidor
    }

    static class producer extends Thread {
        public void run() { // o método run()
            int item;
            while (true) { // laço do produtor
                item = produce_item();
                mon.insert(item);
            }
        }
        private int produce_item() {
            // método para produzir um item
        }
    }

    static class consumer extends Thread {
        public void run() { // o método run()
            int item;
            while (true) { // laço do consumidor
                item = mon.remove();
                consume_item(item);
            }
        }
        private void consume_item(int item) {
            // método para consumir um item
        }
    }

    static class our_monitor { // este é o monitor
        private int buffer[] = new int[N];
        private int count = 0, lo = 0, hi = 0; // contadores e índices

        public synchronized void insert(int val) {
            if (count == N) go_to_sleep(); // se o buffer estiver cheio, va dormir
            buffer[hi] = val; // insere um item no buffer
            hi = (hi + 1) % N; // lugar para colocar o proximo item
            count = count + 1; // mais um item no buffer agora
            if (count == 1) notify(); // se o consumidor estava dormindo, acorde-o
        }

        public synchronized int remove() {
            int val;
            if (count == 0) go_to_sleep(); // se o buffer estiver vazio, va dormir
            val = buffer[lo]; // busca um item no buffer
            lo = (lo + 1) % N; // lugar de onde buscar o proximo item
            count = count - 1; // um item a menos no buffer
            if (count == N - 1) notify(); // se o produtor estava dormindo, acorde-o
            return val;
        }

        private void go_to_sleep() { try{wait();} catch(InterruptedException exc) {};}
    }
}

```

```

static class our_monitor { // este é o monitor
    private int buffer[] = new int[N];
    private int count = 0, lo = 0, hi = 0; // contadores e índices

    public synchronized void insert(int val) {
        if (count == N) go_to_sleep(); // se o buffer estiver cheio, va dormir
        buffer[hi] = val; // insere um item no buffer
        hi = (hi + 1) % N; // lugar para colocar o proximo item
        count = count + 1; // mais um item no buffer agora
        if (count == 1) notify(); // se o consumidor estava dormindo, acorde-o
    }

    public synchronized int remove() {
        int val;
        if (count == 0) go_to_sleep(); // se o buffer estiver vazio, va dormir
        val = buffer[lo]; // busca um item no buffer
        lo = (lo + 1) % N; // lugar de onde buscar o proximo item
        count = count - 1; // um item a menos no buffer
        if (count == N - 1) notify(); // se o produtor estava dormindo, acorde-o
        return val;
    }

    private void go_to_sleep() { try{wait();} catch(InterruptedException exc) {};}
}

```

Outras abordagens para exclusão mútua

Troca de mensagens

Esse algo mais é a troca de mensagens!

Um método de comunicação entre processos baseado em duas primitivas, **send** e **receive**. Que são realizadas via chamadas de sistema, pois não são apenas construções de linguagem. Podem ser inseridas como rotinas de biblioteca, como **send(destination, &message)**; e **receive(source, &message)**;

send: envia uma mensagem para determinado destino.

receive: recebe uma mensagem de uma dada fonte (ou de ANY – qualquer – se o receptor não se importar).

Se nenhuma mensagem estiver disponível, o receptor pode bloquear até que uma chegue. Ou, ele pode retornar imediatamente com um código de erro.

Problemas clássicos de IPC

Conteúdo externo relacionado

1. O problema do produtor-consumidor
2. O problema do jantar dos filósofos
3. O problema dos leitores e escritores

Estes são os três dos problemas mais conhecidos sobre no tema Comunicação entre Processos. Contudo, a literatura de sistemas operacionais está repleta de problemas interessantes que foram amplamente discutidos e analisados usando variados métodos de sincronização.

Introdução aos Impasses (*Deadlocks*)

O problema do produtor-consumidor

Dois processos compartilham de um *buffer* de tamanho fixo comum. Um deles, o produtor, insere informações no *buffer*, e o outro, o consumidor, retira as informações dele.

Um problema didático para elucidar questões com o uso de primitivas de bloqueantes. Também será utilizado para apresentar outras abordagens de comunicação entre processos.

Também conhecido como problema do *buffer* limitado.

É possível generalizar o problema para ter m produtores e n consumidores.

→ Para simplificar, analisaremos apenas o caso de um produtor e um consumidor.

Introdução aos Impasses (*Deadlocks*)

O problema do produtor-consumidor

→ **Produtor:** se o número máximo de itens que o *buffer* pode conter é N , o código do produtor primeiro testará para ver se *count* é N . Se for, o produtor vai dormir, caso contrário o produtor acrescenta um item e incrementa *count*.

→ **Consumidor:** primeiro testa *count* para ver se é 0. Se for, vai dormir; se não for, remove um item e decrece o contador.

→ **Produtor e Consumidor:** testam para ver se o outro deve ser acordado.

Apresenta os mesmos tipos de condições de corrida que vimos anteriormente, pois usa-se a variável *count* para controlar o número de itens no *buffer*. O código para ambos, produtor e consumidor, é mostrado na **Figura 2.27**.

Introdução aos Impasses (*Deadlocks*)

FIGURA 2.27 O problema do produtor-consumidor com uma condição de corrida fatal.

```

#define N 100                                     /* numero de lugares no buffer */
int count = 0;                                    /* numero de itens no buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item( );
        if (count == N) sleep( );
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep( );
        item = remove_item( );
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}

```

Introdução aos Impasses (*Deadlocks*)

O problema do produtor-consumidor

Como o acesso à *count* não é restrito, eventualmente poderá ocorrer a seguinte situação:

O *buffer* está vazio e o consumidor acabou de ler *count* para ver se é 0. Nesse instante, o **escalador decide parar de executar o consumidor temporariamente e começar a executar o produtor**. O produtor insere um item no *buffer*, incrementa *count* e chama *wakeup* para despertar o consumidor. Como o consumidor não está dormindo (ainda), o sinal de despertar é perdido. Retomada a execução do consumidor, ele testará o valor de *count* que ele leu antes (zero) irá dormir. Cedo ou tarde o produtor preencherá o *buffer* e vai dormir também.

→ **Temos um Impasse (*Deadlock*), pois ambos dormirão para sempre!** ←

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

Impasses

Deadlocks

[Faculdade de Tecnologia de Ourinhos]

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br

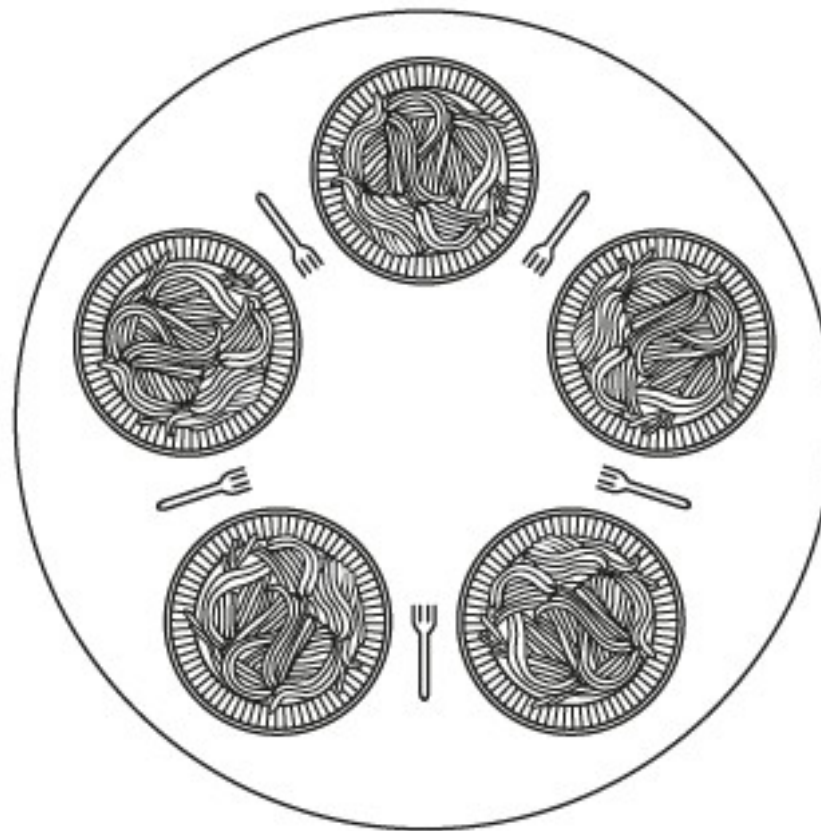
Conceitualização e contexto

Vários recursos envolvidos, sempre.

Muitas vezes, um processo precisa de **acesso exclusivo a vários recursos**.



FIGURA 2.45 Hora do almoço no departamento de filosofia.



Conceitualização e contexto

Vários recursos envolvidos, sempre.

Impasses também podem ocorrer entre máquinas.

Por exemplo, muitos escritórios têm uma rede local com muitos computadores e dispositivos conectados à ela.

Impasses também podem ocorrer em uma série de outras situações.

Por exemplo, em um sistema de banco de dados, por exemplo, um programa pode ter de bloquear vários registros que ele está usando a fim de evitar condições de corrida. Se o processo A bloqueia o registro R1 e o processo B bloqueia o registro R2, e então cada processo tenta bloquear o registro do outro, também teremos um impasse.

→ Impasses podem ocorrer em recursos de hardware ou software ←

Conceitualização e contexto

Recursos

Recursos incluem dispositivos, registros de dados, arquivos e assim por diante. Para tornar a discussão dos impasses mais geral possível, vamos nos referir aos objetos concedidos como recursos.

Acesso aos recursos são o cerne dos impasses. Há dois tipos de recursos:

Preemptível: recurso que **pode** ser retirado do processo proprietário sem **causar-lhe prejuízo** algum. Ex: memória.

Não preemptível: recurso que **não pode** ser tomado do seu proprietário atual, **sem potencialmente causar uma falha**. Ex: Se um processo começou a gravação de um disco, o acesso ao dispositivo não pode ser removido arbitrariamente.

Conceitualização e contexto

Recursos

Em geral, **impasses envolvem recursos não preemptíveis.**

→ Potenciais impasses com recursos preemptíveis, resolvem-se realocando recursos de um processo para outro.

Sequência abstrata de eventos necessários para usar um recurso:

1. Solicitar o recurso.
2. Usar o recurso.
3. Liberar o recurso.

Impasses (*Deadlocks*)

Definição formal

“Um conjunto de processos estará em situação de impasse se cada processo no conjunto estiver esperando por um evento que apenas outro processo no conjunto pode causar.”

TANENBAUM, 2007.

Impasses (*Deadlocks*)

FIGURA 6.2

(a) Código livre de impasses.

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}
```

(a)

(b) Código com impasse potencial.

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}
```

(b)

Codificação

Na Figura 6.2(a), um dos processos adquirirá o primeiro recurso antes do outro. Esse processo então será bem-sucedido na aquisição do segundo recurso e realizará o seu trabalho.

Impasses (*Deadlocks*)

FIGURA 6.2

(a) Código livre de impasses.

(b) Código com impasse potencial.

Codificação

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}
```

(a)

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}
```

(b)

Na Figura 6.2**(b)**, pode ocorrer de o processo A adquirir o recurso 1 e o processo B adquirir o recurso 2. Cada um bloqueará agora quando tentar adquirir o outro recurso. **Nenhum processo executará novamente. Impasse!**

Impasses (*Deadlocks*)

Codificação

O que parece ser apenas uma diferença no estilo de codificação — qual recurso adquirir primeiro — no fim das contas, faz a diferença!

→ O programa pode funcionar ou falhar de uma maneira difícil de ser detectada.

Como todos os processos estão esperando, nenhum deles jamais causará qualquer evento que possa despertar um dos outros membros do conjunto, e todos os processos continuam a esperar para sempre.

Lembre-se da definição formal:

*“Um conjunto de processos estará em situação de impasse se cada processo no conjunto estiver **esperando por um evento que apenas outro processo no conjunto pode causar**.”* TANENBAUM, 2007.

Condições para ocorrência de impasses

Condições necessárias para impasse de recursos (COFFMAN et al., 1971):

- 1. Condição de exclusão mútua.** Cada recurso está atualmente associado a exatamente um processo ou está disponível.
- 2. Condição de posse e espera.** Processos atualmente de posse de recursos que foram concedidos antes podem solicitar novos recursos.
- 3. Condição de não preempção.** Recursos concedidos antes não podem ser tomados à força de um processo. Eles precisam ser explicitamente liberados pelo processo que os têm.
- 4. Condição de espera circular.** Deve haver uma lista circular de dois ou mais processos, cada um deles esperando por um processo de posse do membro seguinte da cadeia.

Condições para ocorrência de impasses

Condições necessárias para impasse de recursos (COFFMAN et al., 1971):

Todas essas quatro condições devem estar presentes para que um impasse de recurso ocorra.

→ Se uma delas estiver ausente, nenhum impasse de recurso será possível.

Lidando com impasses

Como lidar?

Em geral, quatro estratégias são usadas para lidar com impasses.

1. **Simplesmente ignorar o problema.** Se você o ignorar, quem sabe ele ignore você.
2. **Detecção e recuperação.** Deixe-os ocorrer, detecte-os e tome as medidas cabíveis.
3. **Evitar dinamicamente pela alocação cuidadosa de recursos.**
4. **Prevenção, ao negar estruturalmente uma das quatro condições.**



Lidando com impasses

Como lidar – Simplesmente ignorar o problema

A abordagem mais simples é o algoritmo do avestruz: enfie a cabeça na areia e finja que não há um problema.

As pessoas reagem a essa estratégia de maneiras diferentes...

- Matemáticos a consideram inaceitável e dizem que os impasses devem ser evitados a todo custo.
- Engenheiros ponderam a importância deste problema e outros, com base na frequência de ocorrência dos problemas. Se os impasses ocorrem na média uma vez a cada cinco anos, mas quedas do sistema decorrentes de falhas no hardware e defeitos no sistema operacional ocorrem uma vez por semana, a maioria não estaria disposta a pagar um alto preço para eliminar os impasses.

Lidando com impasses

Como lidar – Detecção e recuperação de impasses

Quando essa técnica é usada, o sistema não tenta evitar a ocorrência dos impasses. Em vez disso, ele os deixa ocorrer, tenta detectá-los quando acontecem e então toma alguma medida para recuperar-se após o fato.

- Detecção de impasses com um recurso de cada tipo
- Detecção de impasses com múltiplos recursos de cada tipo

Recuperação de um impasse

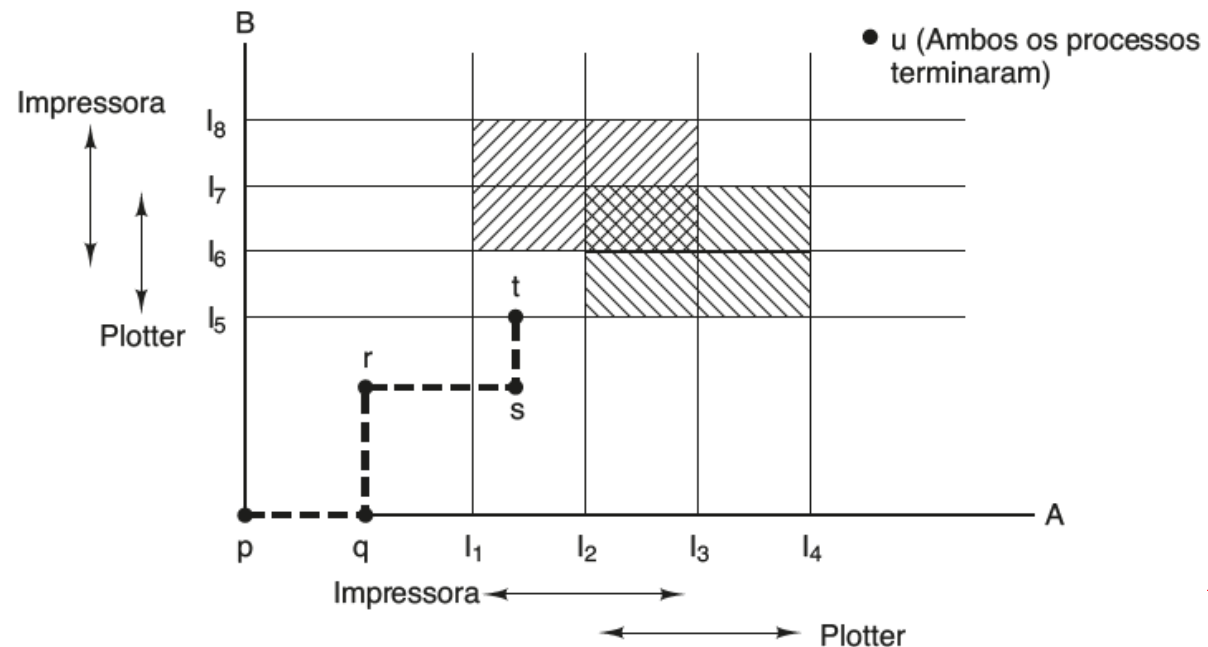
- Recuperação mediante preempção
- Recuperação mediante retrocesso
- Recuperação mediante a eliminação de processos

Lidando com impasses

Como lidar – Evitando impasses

É possível evitar impasses, **se e somente se** determinadas informações estiverem disponíveis. Pois os principais algoritmos para evitar impasses são baseados no conceito de estados seguros. O diagrama de Trajetórias de Recursos ajuda a entender este conceito.

FIGURA 6.8 A trajetória de recursos de dois processos.



Lidando com impasses

Como lidar – Evitando impasses

O Algoritmo do Banqueiro é utilizado para verificar os estados seguros.

Algoritmo de escalonamento desenvolvido por Dijkstra (1965). Ele é modelado da maneira pela qual um banqueiro de uma cidade pequena poderia lidar com um grupo de clientes para os quais ele concedeu linhas de crédito. (Anos atrás, os bancos não emprestavam dinheiro a não ser que tivessem certeza de que poderiam ser ressarcidos.)

O que o algoritmo faz é conferir para ver se conceder a solicitação leva a um estado inseguro:

- Se afirmativo, a solicitação é negada.
- Se conceder a solicitação conduz a um estado seguro, ela é levada adiante.

Lidando com impasses

Como lidar – Prevenindo impasses

Evitar impasses é algo essencialmente impossível, pois isso exige informações a respeito de solicitações futuras, que não são conhecidas.

Então, como os sistemas reais os evitam?

→ **Trabalhando com as quatro condições colocadas por Coffman et al. (1971).**

Se pudermos assegurar que pelo menos uma dessas condições jamais seja satisfeita, então os impasses serão estruturalmente impossíveis (HAVENDER, 1968).

Lidando com impasses

Como lidar – Prevenindo impasses

- 1. Atacando a condição de exclusão mútua.** Se nunca acontecer de um recurso ser alocado exclusivamente para um único processo, jamais teremos impasses.
- 2. Atacando a condição de posse e espera.** Se pudermos evitar que processos que já possuem recursos esperem por mais recursos, poderemos eliminar os impasses.
- 3. Atacando a condição de não preempção.** Realizar a preempção à força, sem prejudicar o processo, é complicado na melhor das hipóteses e impossível no pior cenário.
- 4. Atacando a condição da espera circular.** Ordenar os recursos.

Exercícios

1. Para criarmos softwares que não executam um passo de cada vez, seguindo sempre uma sequência, podemos utilizar um dos conceitos abaixo. Assinale a alternativa que o apresenta.

- a) Sockets.
- b) Árvores Binárias.
- c) Filas.
- d) Threads.
- e) Listas.

1. Para criarmos softwares que não executam um passo de cada vez, seguindo sempre uma sequência, podemos utilizar um dos conceitos abaixo. Assinale a alternativa que o apresenta.

- a) Sockets.
- b) Árvores Binárias.
- c) Filas.
- d) Threads.
- e) Listas.

2. O processador é responsável por obter e armazenar os dados na memória principal quando usa os modos de E/S (Entrada/Saída) programada e por interrupção. A alternativa a esses modos, na qual o módulo de E/S e a memória principal trocam dados diretamente, sem envolvimento do processador, é conhecida como:

- a) E/S Balanceada.
- b) E/S Mapeada na Memória.
- c) E/S Multiplexada.
- d) Arbitração de Barramento.
- e) Acesso Direto à Memória.

2. O processador é responsável por obter e armazenar os dados na memória principal quando usa os modos de E/S (Entrada/Saída) programada e por interrupção. A alternativa a esses modos, na qual o módulo de E/S e a memória principal trocam dados diretamente, sem envolvimento do processador, é conhecida como:

- a) E/S Balanceada.
- b) E/S Mapeada na Memória.
- c) E/S Multiplexada.
- d) Arbitração de Barramento.
- e) Acesso Direto à Memória.

3. Microsoft Windows ou simplesmente Windows é um sistema operacional desenvolvido, comercializado e vendido pela Microsoft. Também é o sistema operacional mais utilizado no mundo, com mais de 90% de participação de mercado. O Windows conta com uma série de aplicativos nativos, ou seja, aplicativos que são instalados juntamente com o Windows quando este é instalado em um computador. Qual das opções não é um aplicativo nativo do Windows?

- a) Paint.
- b) Windows Explorer (explorador de arquivos).
- c) Microsoft Word.
- d) Bloco de Notas.
- e) Nenhuma das alternativas.

3. Microsoft Windows ou simplesmente Windows é um sistema operacional desenvolvido, comercializado e vendido pela Microsoft. Também é o sistema operacional mais utilizado no mundo, com mais de 90% de participação de mercado. O Windows conta com uma série de aplicativos nativos, ou seja, aplicativos que são instalados juntamente com o Windows quando este é instalado em um computador. Qual das opções não é um aplicativo nativo do Windows?

a) Paint.

b) Windows Explorer (explorador de arquivos).

c) Microsoft Word.

d) Bloco de Notas.

e) Nenhuma das alternativas.

4. O sistema operacional como um gerenciador de recursos, gerencia principalmente:

- a) O espaço disponível no disco.
- b) A interface com o usuário.
- c) O teclado.
- d) A Inteligência Artificial que controla o computador.
- e) O processador, a memória e os dispositivos de entrada e saída.

4. O sistema operacional como um gerenciador de recursos, gerencia principalmente:

- a) O espaço disponível no disco.
- b) A interface com o usuário.
- c) O teclado.
- d) A Inteligência Artificial que controla o computador.
- e) O processador, a memória e os dispositivos de entrada e saída.

5. _____ é um termo utilizado na área de ciência da computação para descrever o custo adicional, seja em termos de tempo, recursos ou complexidade, associado à execução de uma determinada operação, processo ou sistema. Esse custo adicional não está diretamente relacionado à funcionalidade principal da operação, mas sim às tarefas adicionais necessárias para executá-la.

Assinale o termo adequado para o preenchimento da lacuna, de modo que o conceito apresentado demonstre-se correto:

- a) Stack Overflow.
- b) Overhead.
- c) Hedge Assimétrico.
- d) Overclock.
- e) Tuning.

5. _____ é um termo utilizado na área de ciência da computação para descrever o custo adicional, seja em termos de tempo, recursos ou complexidade, associado à execução de uma determinada operação, processo ou sistema. Esse custo adicional não está diretamente relacionado à funcionalidade principal da operação, mas sim às tarefas adicionais necessárias para executá-la.

Assinale o termo adequado para o preenchimento da lacuna, de modo que o conceito apresentado demonstre-se correto:

a) Stack Overflow.

b) Overhead.

c) Hedge Assimétrico.

d) Overclock.

e) Tuning.

Sistemas Operacionais I

[Análise e Desenvolvimento de Sistemas – 3º Semestre]

That's all folks!

Prof. Wellington Camargo

wellington.camargo@fatecourinhos.edu.br